МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЖИТОМИРСЬКА ПОЛІТЕХНІКА
Кафедра ММСА

ЛАБОРАТОРНА РОБОТА № 2

з дисципліни «Моделювання та аналіз програмного забезпечення»

ПОБУДОВА ДІАГРАМИ КЛАСІВ ПРЕДМЕТНОЇ ОБЛАСТІ

Варіант предметної області: «Півот» — вебсервіс і браузерне розширення для вивчення іноземної мови на власному контенті користувача

Виконав: студент групи ВТ-23-2
Камінський Олексій Дмитрович

Перевірив: Власенко О.В.

Житомир – 2026

Репозиторій з кодом роботи:
`https://github.com/vt232kod-debug/software-modeling`

Мета

Варіант предметної області: «Півот» — вебсервіс і браузерне розширення для вивчення іноземної мови на основі власного контенту користувача. Роботу виконав студент групи ВТ-23-2 Камінський Олексій Дмитрович.

Опанувати нотацію діаграми класів UML (Unified Modeling Language — уніфікована мова моделювання): навчитися коректно моделювати класи, атрибути, методи та всі базові типи відношень — асоціацію з множинністю, агрегацію, композицію, узагальнення й реалізацію інтерфейсу — для предметної області, зафіксованої в Лабораторній роботі №1.

Предметна область спільна для всіх лабораторних робіт курсу й для курсової роботи, і збігається з темою дипломної роботи. Межі моделі, прийняті в ЛР1, діють без змін: відеоплатформа розглядається одна — YouTube, а джерело субтитрів заховане за інтерфейсом `ДжерелоСубтитрів`, тому інші площадки присутні лише як точка підстановки з позначкою «інші адаптери поза межами роботи»; штучний інтелект — зовнішній сервіс за API, тому в моделі класів він лишає по собі не класи, а сліди роботи (вектори, перекази, теми словника); мобільний застосунок і Safari-версія — поза межами роботи; телеметрії й аналітики в системі свідомо немає.

Одну межу ця робота уточнює окремо, бо вона стосується саме моделі класів: **матеріал-вебсторінка не отримує власного класу-нащадка**. Стаття, додана посиланням, входить у модель як `Документ` з форматом «стаття-html» (UC12) і розбирається реалізацією `ПарсерСтатті`; третій нащадок `Матеріал` для вебсторінки свідомо лишений поза межами роботи, бо він не додав би жодного нового типу відношення — лише третій набір тих самих атрибутів. Тому формулювання «рівно два нащадки» далі в тексті означає «два нащадки в межах цієї моделі», а не «два види матеріалу в системі».

Якщо ЛР1 відповідала на питання «що система робить і для кого», то ця робота відповідає на питання «з чого система складається»: які сутності вона зберігає, що саме вона про них знає і як вони пов’язані життєвим циклом. Це перехід від моделі вимог до моделі даних, і головна перевірка якості тут — трасованість: кожен клас мусить мати походження в конкретному варіанті використання або в текстовій специфікації ЛР1.

Розділ 1. Від Use Case до класів: як виділялися сутності

Класи не вигадувалися наново. Джерелом був єдиний перелік із ЛР1: 18 варіантів використання, три повні текстові специфікації (UC04, UC05, UC12) і таблиця відкинутих кандидатів. Застосовано три послідовні фільтри.

Фільтр 1 — іменник зі специфікації, що має власний стан. З основних і альтернативних потоків UC04, UC05 і UC12 виписано всі іменники. Далі для кожного ставилося питання: чи система **пам'ятає** його між сеансами? «Доріжка реплік» — пам'ятає, бо кешує за відбитком: отже, `Транскрипт`. «Режим показу субтитрів» — пам'ятає, але це значення налаштування, а не сутність: отже, рядок `НалаштуванняКористувача`, а не клас. «Крок назад клавішею» — не пам'ятає взагалі: отже, не клас і не атрибут, а поведінка інтерфейсу.

Фільтр 2 — власний життєвий цикл. Іменник ставав класом, лише якщо він створюється й видаляється окремо від інших. `СтанПовторення` має власний життєвий цикл усередині життя картки слова, тому це клас, пов'язаний композицією, а не група полів у `ЗбереженеСлово`: стан перераховується після кожної вправи, і його зручно читати окремо від самої картки. Навпаки, `Реплика` власного життєвого циклу не має — вона народжується й помирає разом із доріжкою, не має ідентичності й не посилається ні на що, — тому лишилася типом значення в атрибуті `Транскрипт.реплики: Реплика[]`.

Фільтр 3 — сутність, а не процес. Назви на «-ер», «-ор», «-менеджер», «-сервіс» на діаграмі класів предметної області не місце: це компоненти, і їхнє місце — ЛР3. Єдиний виняток свідомо зроблено для двох інтерфейсів і їхніх реалізацій (`ДжерелоСубтитрів`, `ПарсерДокумента`): методичка прямо вимагає показати реалізацію інтерфейсу, а саме ці дві точки підстановки є головними архітектурними рішеннями теми й безпосередньо породжують класи предметної області — `Транскрипт` і `Документ` відповідно.

Висновок: три фільтри дали 18 класів предметної області (серед них один абстрактний і один резервний), два інтерфейси й три класи оточення інтерфейсів — дві реалізації та фабрику; усе, що фільтри відкинули, не зникло, а перейшло в атрибути, у типи значень або в майбутні компоненти ЛР3.

Розділ 2. Перелік класів предметної області

Методичка вимагає щонайменше 6–8 класів, включно з одним абстрактним класом і одним інтерфейсом. Фактично побудовано **23 елементи**: 17 основних класів

предметної області (серед них обов’язковий абстрактний `Матеріал`), 1 резервний клас, 2 інтерфейси, 2 класи-реалізації і 1 фабрику.

№	Елемент	Тип	Призначення
1	`Користувач`	Клас	Власник усіх особистих даних: акаунт, його словник, матеріали, профіль лексики, тариф
2	`НалаштуванняКористувача`	Клас	Налаштування як пара «ключ — значення»: одне джерело істини для вебзастосунку й розширення
3	`Матеріал`	Абстрактний клас	Будь-яка одиниця контенту в історії: те спільне, за що користувач бачить матеріал у списку
4	`Відео`	Клас (нащадок `Матеріал`)	Відеоматеріал платформи з тривалістю й позицією перегляду в мілісекундах
5	`Документ`	Клас (нащадок `Матеріал`)	Власний текст користувача в читалці: формат, вміст, сторінки, відпечаток
6	`ЗбереженеСлово`	Клас	Картка слова, збережена з матеріалу разом із реплікою-контекстом і мовною парою
7	`ТемаСловника`	Клас	Смислова тема словника, запропонована ШІ; службова тема «Різне» для негрупованого
8	`ВідомеСлово`	Клас	Запис профілю відомої лексики, окремого для кожної мови
9	`СтанПовторення`	Клас	Стан картки в розписанні інтервальних повторень (FSRS)
10	`ОцінкаРівня`	Клас	Результат тесту обсягу словника: «≈N слів»,

			похибка, рівень CEFR
11	`Підписка`	Клас	Платний стан акаунта: провайдер, зовнішній ідентифікатор, статус, кінець періоду
12	`ЛічильникВикористання`	Клас	Денна витрата лімітів за видами (переклад, пояснення, розбір, переказ, розпізнавання)
13	`СловниковаСтаття`	Клас	Спільна стаття довідника: лема, пара мов, значення, частини мови
14	`Транскрипт`	Клас	Доріжка реплік відео. Спільно зберігається доріжка, зроблена розпізнаванням; доріжку, віддану самою платформою, розширення читає в межах сторінки
15	`РоботаРозпізнавання`	Клас	Замовлення доріжки в сервісу ШІ: стан, етап, кількість спроб, тривалість звуку (UC10)
16	`КешПерекладу`	Клас	Спільний переклад доріжки за відбитком «мови + рядки»: один переклад обслуговує всіх глядачів ролика
17	`КешПоясснень`	Клас	Спільна відповідь ШІ за відбитком запиту: пояснення слова в контексті (UC08) і розбір речення (UC09)
18	`ПереказДоріжки`	Клас (резервний)	Спільний переказ доріжки за відбитком її рядків
19	`ДжерелоСубтитрів`	Інтерфейс	Контракт платформенного адаптера: плеєр, навігація, доріжка, розкладка панелі
20	`ДжерелоСубтитрівYou	Клас-реалізація	Єдина реалізація `ДжерелоСубтитрів` у

	Tube`		межах роботи
21	`ФабрикаДжерелСубтитрів`	Клас	Обирає реалізацію за платформою (фабричний метод до стратегії)
22	`ПарсерДокумента`	Інтерфейс	Контракт розбору файла в `Документ`: підтримка формату, розбір, пагінація, мова
23	`ПарсерPDF`	Клас-реалізація	Єдиний парсер, що дає справжні межі сторінок оригіналу

Клас `ПереказДоріжки` в моделі предметної області був позначений як резервний, і його свідомо внесено на діаграму: без нього з таблиці відношень зникає асоціація «Транскрипт — ПереказДоріжки». Разом із `Транскрипт`, `СловниковаСтаття`, `РоботаРозпізнавання`, `КешПерекладу` і `КешПоясснень` він утворює шостий за розміром, але найцікавіший змістовно пакет — спільні довідники без власника, який окремо пояснено в Розділі 6.1.

Три класи — `РоботаРозпізнавання`, `КешПерекладу` і `КешПоясснень` — додано до моделі саме тому, що без них варіанти використання UC08, UC09 і UC10 трасувалися б лише на методи й лічильники, тоді як їхній результат система **пам’ятає** між сеансами, а це і є Фільтр 1 з Розділу 1. `КешПоясснень` свідомо зведено в один клас для двох видів розбору (пояснення слова й будова речення), бо відрізняються вони лише складом ключа й текстом запиту, а не типами відношень.

2.1. Абстрактний клас `Матеріал` і чому він абстрактний

`Матеріал` оголошено абстрактним, бо матеріал ніколи не існує «сам по собі»: у системі він завжди або **Відео** (є платформа, зовнішній ідентифікатор, тривалість, позиція в мілісекундах), або **Документ** (є формат, вміст, межі сторінок, відпечаток вмісту). Спільним лишається рівно те, за що користувач бачить матеріал у своїй історії: назва, мова контенту, карта частот слів, оцінка зрозумілості за його профілем лексики й позиція читання.

Ключовим є абстрактний метод `джерелоТексту(): Iterable<Реплика>`. У відео це репліки з часовими мітками, у документі — сторінки, розбиті на речення, але вся решта системи працює з ними однаково: оцінка зрозумілості, виділення фрагмента, збереження слова з контекстом, розбір речення. Якби `Матеріал` не був абстрактним, довелося б або дублювати обчислення зрозумілості у двох класах,

або створювати «просто матеріал» без джерела тексту — об’єкт, із яким нічого не можна зробити.

Це узагальнення не є теоретичною красою: воно **узгоджене** зі схемою даних, де обидва види лежать в одній таблиці історії з перелічуванням платформи, а документ додатково має власний рядок з деталями формату й вмісту. Слово «узгоджене», а не «тотожне», тут вжите свідомо: схема знає третє значення платформи для сторінки-статті, і модель приймає її як `Документ` з форматом «стаття-html», а не як третього нащадка — межу зафіксовано в розділі «Мета». Друге розходження того самого роду — кратності з боку власника: у схемі власник рядка історії може бути ще не прив’язаний, і модель показує це кратністю `0..1` (Розділ 6).

Нащадків у `Матеріал` рівно два, і підставою є не «у системі лише два види контенту», а правило поділу: нащадок з’являється там, де відрізняється **реалізація** `джерелоТексту()`. У відео джерелом є доріжка реплік із часовими мітками, у документі — розбитий на сторінки текст. Стаття за посиланням дає рівно той самий текст, що й завантажений файл, тому власної реалізації не потребує й нащадком не стає.

2.2. Два інтерфейси й чому саме вони

`ДжерелоСубтитрів` — головне архітектурне рішення теми дипломної роботи. Інтерфейс оголошує шість операцій: `знайтиПлеєр()`, `спостерігатиНавігацію()`, `сховатиРодніСубтитри()`, `отриматиДоріжку()`, `побудуватиДжерелоСлова()`, `розкладкаПанелі()`. Реалізацію обирає `ФабрикаДжерелСубтитрів` за платформою — це патерн Strategy з фабричним методом. Уся решта розширення платформенно-сліпа: оверлей, переклад і сховище знають лише інтерфейс. Саме тому обмеження «лише YouTube» у цій роботі — архітектурне рішення про межі моделі, а не вимушене спрощення: `ДжерелоСубтитрівВласнийФайл` (імпорт файлу `.srt`, `.vtt`, `.ass`, `.ttml`), `ДжерелоСубтитрівРозпізнане` (доріжка, зроблена ШІ, — UC10) та адаптери інших платформ входять тим самим каналом. Серверний бік UC10 — це окремий клас `РоботаРозпізнавання`: саме він зберігає стан замовлення, тоді як `ДжерелоСубтитрівРозпізнане` лише читає готову доріжку тим самим контрактом і тому на повну діаграму не внесений.

`ПарсерДокумента` — друга точка підстановки, що безпосередньо походить із кроків 4–5 основного потоку UC12: «система обирає парсер за розширенням і MIME-типом файла», «парсер витягує текст і, для PDF, справжні межі сторінок оригіналу». Інтерфейс оголошує `підтримує()`, `розібрати()`, `розбитиНаСторінки()`,

`визначитиМову()`, а реалізації — `ПарсерPDF`, `ПарсерEPUB`, `ПарсерDOCX`, `ПарсерFB2`, `ПарсерПростогоТексту` (TXT, MD і вставлений текст), `ПарсерСтатті` (HTML за посиланням). Новий формат додається без жодної зміни в читалці — саме це й перевіряє альтернативний потік 4а UC12 «формат не підтримується жодним парсером».

Два інтерфейси, а не один, узято тому, що вони реалізують різні контракти в різних частинах системи: перший живе в браузері користувача, другий — на сервері. Жоден із них не є узагальненням другого, і спільного предка в них немає — як і в прикладі методички, де інтерфейс `Оцінюваний` реалізують непов'язані спадкуванням класи.

2.3. Що свідомо не стало класом

Кандидат	Чому не клас	Куди перейшов
`Реплика`	Тип значення без ідентичності: народжується й помирає разом із доріжкою, не посилається ні на що й не змінюється окремо	Атрибут `Транскрипт.реплики: Реплика[]`, тип повернення `реплікаНаЧас(мс)` і `джерелоТексту()`
`КарткаСлова` (UI-картка з UC05)	Екранне подання, а не сутність: та сама картка збирається зі `СловниковаСтаття` або з машинного перекладу й нічого не зберігає	Методи `ЗбереженеСлово.озвучити()`, `СловниковаСтаття.головніПереклади()`
`МенеджерСловника`, `СервісПерекладу`, `КонтролерЧиталки`	Процеси, а не сутності предметної області: у них немає стану, який система пам’ятає	Компоненти ЛР3 («Модуль навчання», «Шар зовнішніх сервісів», «Веб-інтерфейс»)
`Мова`	Довідник із кількох десятків нерухомих рядків; окремий клас не додає ні поведінки, ні зв’язків	Тип-код `КодМови` в атрибутах `мовaКонтенту`, `мовaДжерела`, `мовaЦілі`, `мова`
`ДенніЛіміти` як окремий клас	Ліміт — не запис, а функція тарифу й виду витрати	Методи `Підписка.стеля(вид)` і `ЛічильникВикористання.залишок(тариф)`
`ІсторіяМатеріалів`	Колекція, а не сутність: списком керує кратність `0..*` на композиції	Композиція `Користувач ◆— Матеріал` з роллю «історія матеріалів»

Висновок: перелік містить 23 елементи діаграми, серед них обов’язкові за методичкою абстрактний клас `Матеріал` і два інтерфейси; шість кандидатів свідомо відкинуто, бо вони були або типами значень, або процесами, або колекціями, а не сутностями з власним станом і життєвим циклом.

Розділ 3. Атрибути, методи й модифікатори видимості

Методичка вимагає щонайменше 3 атрибути з коректною видимістю й типом і 2 методи на клас. Вимогу витримано для всіх 21 класу діаграми: найскромніші за складом — `ЛічильникВикористання`, `ФабрикаДжерелСубтитрів` і `ПарсерPDF` (по 3 атрибути й 2–3 методи), найповніші за атрибутами — `ЗбереженеСлово`, `Транскрипт` і `РоботаРозпізнавання` (по 7), найповніший за операціями — `ЗбереженеСлово` (7 методів). Усього на діаграмі 106 атрибутів і 73 методи. Два інтерфейси атрибутів не мають за означенням — вони оголошують лише операції (6 і 4 відповідно).

3.1. Правило вибору видимості

Видимість проставлялася не навмання, а за трьома правилами, однаковими для всієї діаграми.

Приватний (`-`) — усе, що є технічним ідентифікатором або похідним станом. Сюди потрапили всі `id: UUID` (крім `ЛічильникВикористання`, у якого технічного ідентифікатора немає зовсім: рядок ідентифікується парою «день + вид витрати» в межах власника, і обидва атрибути пари позначено стандартним UML-модифікатором `{id}`), усі службові дати (`створений`, `додано`, `пройдено`, `відкрита`, `оновлено`), а також обчислювані кеші: `Матеріал.картаЧастот`, `Матеріал.кількістьТокенів`, `ЗбереженеСлово.вектор`, `Документ.початкиСторінок`, `Документ.вміст`. Жодне з цих полів не має сенсу змінювати ззовні: карта частот перераховується при завантаженні матеріалу, вектор — при збереженні слова, межі сторінок — при розборі файла. Окремо приватним зроблено `Користувач.поштаПідтверджена`: цей прапорець має право змінити лише сценарій підтвердження пошти.

Публічний (`+`) — усе, що користувач бачить або чим керує через інтерфейс. `назва`, `слово`, `переклад`, `контекст`, `формат`, `поточнаСторінка`, `статус`, `кінецьПеріоду`, `cefr`, `обсягСловника`. Критерій простий: якщо значення з’являється на екрані в сценаріях ЛР1, воно публічне.

Захищений (`#`) — лише те, що успадковують нащадки абстрактного класу. Такий атрибут на діаграмі рівно один: `Матеріал.id: UUID`. Він має бути доступним у `Відео` й `Документ` (обидва будують за ним посилання на запис історії), але не має бути доступним іззовні. Це й є практична ілюстрація різниці між `-` і `#`: приватний ідентифікатор `Користувач.id` нащадків не має й тому лишається приватним, а `Матеріал.id` без `#` зробив би успадкування безглуздим.

Методи всюди публічні (`+`): на рівні моделі предметної області показується контракт сутності, а не її внутрішня кухня. Абстрактний метод позначено явно —

`+ {abstract} джерелоТексту(): Iterable<Реплика>`; PlantUML рендерить його курсивом, як і назву абстрактного класу.

3.2. Перелічувані типи, ужиті в атрибутах

Щоб не захаращувати діаграму, перелічувані типи (enum) не виносилися в окремі елементи, а використані як типи атрибутів. Нижче — їхні множини значень; усі вони походять із моделі предметної області та зі специфікацій ЛР1.

Тип	Значення	Де вжито
`ТарифEnum`	базовий, з підпискою	`Користувач.тариф()`, `ЛічильникВикористання.залишок()`
`СтатусEnum`	пробна, активна, прострочена, скасована	`Підписка.статус`
`ВидВитратиEnum`	переклад, пояснення, розбір, переказ, розпізнавання, завантаження	`ЛічильникВикористання.вид`, `Підписка.стеля()`
`ПлатформаEnum`	youtube, document (інші адаптери поза межами роботи)	`Відео.платформа`, `Транскрипт.платформа`, `ФабрикаДжерелСубтитрів`
`ФорматEnum`	pdf, epub, docx, fb2, txt, md, стаття-html	`Документ.формат`, `ПарсерДокумента.підтримує()`
`ОдиницяEnum`	сторінка оригіналу, фіксована кількість знаків, абзац	`Документ.одиницяСторінки`
`СтанEnum`	нова, вчиться, повторення, перевчається, відкладена	`СтанПовторення.стан`
`ПоходженняEnum`	пряма відмітка користувача, виведене за частотним рангом	`ВідомеСлово.походження`
`ВидРозборуEnum`	пояснення слова в контексті, розбір будови речення	`КешПоясснень.вид`
`АкцентEnum`	варіанти вимови для озвучки, обрані в мастері першого запуску	`ЗбереженеСлово.озвучити()`
`СтанРоботиEnum`	у черзі, звук, модель, запис, готово, помилка	`РоботаРозпізнавання.стан`
`РезультатВправи`	успішний, невдалий	`СтанПовторення.застосуватиВідповідь()`
`ФорматСубтитрівEnum`	srt, vtt, ass, ttml	`ДжерелоСубтитрівВласнийФайл.формат` — клас поза повною діаграмою, показаний сірим лише на частковому виді 3

Два перелічення знадобляться вже в наступних роботах: `СтанEnum` і `СтанРоботиEnum` — це готові набори станів для діаграм станів картки слова й роботи з

розпізнавання мовлення. Усі перелічення, крім `ФорматСубтитрівEnum`, ужиті в атрибутах класів, що справді присутні на повній діаграмі (рисунок 1); `ФорматСубтитрівEnum` належить класові поза межами роботи й тому позначений окремо.

Два значення в таблиці виділено **напівжирним**: вони додані за результатами Лабораторних робіт №5 і №6 і в первісній редакції цієї роботи відсутні. Значення `відкладена` в `СтанEnum` знадобилося тому, що діаграма станів ЛР5 зробила відкладання картки окремим підстаном її життєвого циклу, а атрибут `СтанПовторення.стан` мусить уміти зберігати й це значення. Перелічуваний тип `РезультатВправи` у первісній редакції фігурував лише як тип параметра `застосуватиВідповідь(результат: РезультатВправи)`, і склад його значень не було розкрито; охоронні умови `[результат.успішний]` і `[не результат.успішний]` на восьми переходах ЛР5 читаються саме як порівняння з цими двома значеннями. Підстави обох доповнень зведено в Розділі 9.

Висновок: усі 21 клас діаграми мають щонайменше 3 атрибути й 2 методи, видимість проставлено за трьома явними правилами, а єдиний захищений атрибут `Матеріал.id` показує, навіщо модифікатор `#` взагалі існує.

Розділ 4. Повна діаграма класів

Діаграму побудовано в PlantUML, файл `classes.puml`; рендеринг — локальна збірка Graphviz. Елементи згруповано в п’ять пакетів, що відповідають природним підсистемам: «Акаунт і доступ», «Матеріали користувача», «Словник і прогрес», «Спільні довідники без власника», «Підстановка джерел і парсерів». Пакети — лише засіб читабельності; жодного відношення між пакетами в UML-сенсі не оголошено.

На діаграмі присутні **усі п’ять типів відношень** із розділу 1.2 методички: 9 композицій, 2 агрегації, 7 асоціацій з явною множинністю, 2 узагальнення й 2 реалізації інтерфейсу, а також 4 залежності — разом 26 зв’язків.

```
@startuml
' Лабораторна робота №2. Діаграма класів предметної області
' Предметна область: «Півот» — вебсервіс і браузерне розширення
' для вивчення іноземної мови на основі власного контенту користувача
' Камінський Олексій Дмитрович, група ВТ-23-2


skinparam defaultFontName "Helvetica"
skinparam defaultFontSize 15
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam packageStyle rectangle
skinparam ArrowFontSize 13
skinparam nodesep 22
```

```
skinparam ranksep 55
skinparam class {
  BackgroundColor #FEFEFE
  BorderColor #33668E
  ArrowColor #33668E
}
skinparam package {
  BackgroundColor #F7FAFC
  BorderColor #9AB3C6
  FontStyle bold
}
skinparam interface {
  BackgroundColor #FFF8EC
  BorderColor #B07A28
}
skinparam note {
  BackgroundColor #FFFDF2
  BorderColor #C9B47A
}

' ========================================================================
' ПАКЕТ 1. Акаунт і доступ
' ========================================================================
package "Акаунт і доступ" {


  class Користувач {
    - id: UUID
    + пошта: Email
    + ім'я: String
    - поштаПідтверджена: Boolean
    - створений: DateTime
    + тариф(): ТарифEnum
    + профільВідомих(мова: КодМови): Set<ВідомеСлово>
    + вивантажитиДані(): Архів
  }


  class НалаштуванняКористувача {
    - id: UUID
    + {field} ключ: String(64)
    + значення: JSON
    - оновлено: DateTime
    + встановити(ключ: String, значення: JSON): void
    + секретний(): Boolean
  }


  class Підписка {
    - id: UUID
    + провайдер: String
    + зовнішнійId: String
    + статус: СтатусEnum
    + кінецьПеріоду: DateTime
    + активна(): Boolean
    + пробна(): Boolean
    + стеля(вид: ВидВитратиEnum): Int
  }


  class ЛічильникВикористання {
    + день: Date {id}
    + вид: ВидВитратиEnum {id}
    + кількість: Int
    + списати(n: Int): void
    + залишок(тариф: ТарифEnum): Int
    + скиданняЧерез(): Duration
  }
}
```

```
' ========================================================================
' ПАКЕТ 2. Матеріали користувача
' ========================================================================
package "Матеріали користувача" {


  abstract class Матеріал {
    # id: UUID
    + назва: String
    + моваКонтенту: КодМови
    + зрозумілість: Real
    - картаЧастот: Map<String,Int>
    - кількістьТокенів: Int
    + перерахуватиЗрозумілість(профіль: Set<ВідомеСлово>): Real
    + позиціяЧитання(): Progress
    + {abstract} джерелоТексту(): Iterable<Реплика>
  }


  class Відео {
    + платформа: ПлатформаEnum
    + зовнішнійId: String
    + тривалістьМс: Long
    + останняПозиціяМс: Long
    + джерелоТексту(): Iterable<Реплика>
    + посиланняНаМомент(мс: Long): URL
  }


  class Документ {
    + формат: ФорматEnum
    - вміст: Text
    - початкиСторінок: Int[]
    + відпечаток: SHA256
    + поточнаСторінка: Int
    + одиницяСторінки: ОдиницяEnum
    + джерелоТексту(): Iterable<Реплика>
    + сторінка(n: Int): Text
    + змінитиМову(код: КодМови): void
  }
}


' ========================================================================
' ПАКЕТ 3. Словник, профіль лексики та прогрес
' ========================================================================
package "Словник і прогрес" {


  class ЗбереженеСлово {
    - id: UUID
    + слово: String
    + переклад: String
    + контекст: String
    + моваДжерела: КодМови
    + моваЦілі: КодМови
    - {field} вектор: Vector(768)
    + {static} створити(слово: String, переклад: String, контекст: String, моваДжерела:
КодМови, моваЦілі: КодМови): ЗбереженеСлово
    + озвучити(акцент: АкцентEnum): Audio
    + схожіЗаЗмістом(n: Int): List<ЗбереженеСлово>
    + прикріпитиДоТеми(тема: ТемаСловника): void
    + зарахуватиДоЗнайомих(): ВідомеСлово
    + позначитиВідомим(): ВідомеСлово
    + видалити(): void
  }


  class ТемаСловника {
    - id: UUID
    + назва: String
    + піктограма: String
```

```
    + службова: Boolean
    - відкрита: DateTime
    + прогрес(): Real
    + нові(): Int
    + перейменувати(назва: String): void
  }

  class ВідомеСлово {
    - id: UUID
    + слово: String
    + мова: КодМови
    + походження: ПоходженняEnum
    - додано: DateTime
    + скасувати(): void
    + виведене(): Boolean
  }

  class СтанПовторення {
    - id: UUID
    + наступне: DateTime
    + стабільність: Real
    + складність: Real
    + повторень: Int
    + стан: СтанEnum
    + застосуватиВідповідь(результат: РезультатВправи): void
    + прострочене(): Boolean
    + наступнийІнтервал(): Duration
    + короткийКрок(): Duration
    + відкласти(): void
    + продовжити(): void
  }

  class ОцінкаРівня {
    - id: UUID
    + мова: КодМови
    + обсягСловника: Int
    + cefr: String
    - пройдено: DateTime
    + похибка(): Int
    + виведеніВідомі(корпус: ЧастотнийКорпус): List<String>
    + динаміка(): Delta
  }
}

' ====================================================================
' ПАКЕТ 4. Спільні довідники без власника
' ====================================================================
package "Спільні довідники без власника" {

  class Транскрипт {
    - id: UUID
    + платформа: ПлатформаEnum
    + відеоId: String
    + мова: КодМови
    + реплики: Репліка[]
    + кількістьРеплік: Int
    + постачальник: String
    + реплікаНаЧас(мс: Long): Репліка
    + відпечаток(): SHA256
    + перекласти(цільоваМова: КодМови): Транскрипт
  }

  class РоботаРозпізнавання {
    - id: UUID
    + платформа: ПлатформаEnum
    + відеоId: String
```

```
    + стан: СтанРоботиEnum
    - етап: String
    + спроб: Int
    + тривалістьЗвукуМс: Long
    + просунути(стан: СтанРоботиEnum): void
    + завершена(): Boolean
    + причинаНеудачі(): String
  }

  class КешПерекладу {
    - ключ: SHA256
    + моваДжерела: КодМови
    + моваЦілі: КодМови
    + переклади: String[]
    + кількістьРядків: Int
    + постачальник: String
    + знайти(ключ: SHA256): String[]
    + зберегти(ключ: SHA256, переклади: String[]): void
    + частковий(): Boolean
  }

  class КешПояснень {
    - ключ: SHA256
    + вид: ВидРозборуEnum
    + запит: String
    + відповідь: JSON
    + постачальник: String
    - востаннєВжито: DateTime
    + знайти(ключ: SHA256): JSON
    + зберегти(ключ: SHA256, відповідь: JSON): void
    + застарілий(): Boolean
  }

  class ПереказДоріжки {
    - ключ: SHA256
    + назва: String
    + переказ: JSON
    + кількістьРядків: Int
    + рядки(): List<String>
    + збігаєтьсяЗ(т: Транскрипт): Boolean
  }

  class СловниковаСтаття {
    - id: UUID
    + лема: String
    + параМов: String
    + важливість: Real
    + значення: Значення[]
    + головніПереклади(n: Int): List<String>
    + дляБудьЯкоїФорми(форма: String): Boolean
  }
}

' ========================================================================
' ПАКЕТ 5. Точки підстановки: інтерфейси та їхні реалізації
' ========================================================================
package "Підстановка джерел і парсерів" {

  interface ДжерелоСубтитрів {
    + знайтиПлеєр(): Плеєр
    + спостерігатиНавігацію(обробник: Обробник): void
    + сховатиРодніСубтитри(прапор: Boolean): void
    + отриматиДоріжку(): Транскрипт
    + побудуватиДжерелоСлова(позиціяМс: Long): ДжерелоСлова
    + розкладкаПанелі(): Розкладка
  }
```

```
  class ДжерелоСубтитрівYouTube {
    - відеоId: String
    - адресаTimedtext: URL
    - автогенерована: Boolean
    - елементПлеєра: Плеєр
    + отриматиДоріжку(): Транскрипт
    + сховатиРодніСубтитри(прапор: Boolean): void
    + прийнятиДоріжку(json: JSON): Транскрипт
    .. решта операцій інтерфейсу ..
  }

  class ФабрикаДжерелСубтитрів {
    - реєстр: Map<ПлатформаEnum,Тип>
    - типоваПлатформа: ПлатформаEnum
    - активне: ДжерелоСубтитрів
    + створити(платформа: ПлатформаEnum): ДжерелоСубтитрів
    + зареєструвати(платформа: ПлатформаEnum, тип: Тип): void
  }

  interface ПарсерДокумента {
    + підтримує(формат: ФорматEnum): Boolean
    + розібрати(файл: Файл): Документ
    + розбитиНаСторінки(текст: Text): Int[]
    + визначитиМову(текст: Text): КодМови
  }

  class ПарсерPDF {
    - максимальнийРозмірМБ: Int
    - маєТекстовийШар: Boolean
    - межіСторінокОригіналу: Int[]
    + розібрати(файл: Файл): Документ
    + розбитиНаСторінки(текст: Text): Int[]
    .. решта операцій інтерфейсу ..
  }
}

' ========================================================================
' УЗАГАЛЬНЕННЯ (generalization)
' ========================================================================
Матеріал <|-- Відео
Матеріал <|-- Документ

' ========================================================================
' РЕАЛІЗАЦІЯ ІНТЕРФЕЙСУ (realization)
' ========================================================================
ДжерелоСубтитрів <|.. ДжерелоСубтитрівYouTube
ПарсерДокумента <|.. ПарсерPDF

' ========================================================================
' КОМПОЗИЦІЯ (composition) — спільний життєвий цикл з акаунтом
' ========================================================================
Користувач "0..1" *-- "1..*" НалаштуванняКористувача : налаштування
Користувач "1" *-- "0..1" Підписка : тариф
Користувач "1" *-- "0..*" ЛічильникВикористання : денні витрати
Користувач "0..1" *-- "0..*" Матеріал : історія матеріалів
Користувач "0..1" *-- "0..*" ЗбереженеСлово : особистий словник
Користувач "1" *-- "0..*" ТемаСловника : теми словника
Користувач "0..1" *-- "0..*" ВідомеСлово : профіль лексики
Користувач "0..1" *-- "0..*" ОцінкаРівня : історія тестів
ЗбереженеСлово "1" *-- "0..1" СтанПовторення : розписання


note bottom of Користувач
  **0..1, а не 1, у п'яти композиціях:**
```

```
  рядок, накопичений до першого входу,
  ще не має власника. Нуль — перехідний
  стан до прив'язки, а не другий власник.
  Де власник обов'язковий (підписка,
  лічильники, теми) — кратність 1.
  Докладно: Розділ 6, вступ.
end note


' ====================================================================
' АГРЕГАЦІЯ (aggregation) — частина не належить цьому цілому як композит
' ====================================================================
Матеріал "1" o-- "0..*" ЗбереженеСлово : джерело слова
note on link
  **Агрегація, а не композиція — компроміс.**
  Кратність 1: картки слова без матеріалу
  не буває. Частина цього цілого НЕ переживає:
  видалення матеріалу забирає й слова,
  й розписання. Ромб порожній лише тому, що
  композит у частини один — Користувач.
  Розділ 7, рядок 10.
end note


ТемаСловника "0..1" o-- "0..*" ЗбереженеСлово : згруповані слова


' ====================================================================
' АСОЦІАЦІЯ З КРАТНІСТЮ (association)
' ====================================================================
Відео "0..*" -- "0..1" Транскрипт : доріжка реплік >
Відео "0..*" -- "0..1" РоботаРозпізнавання : замовлене розпізнавання >
Транскрипт "1" -- "0..*" КешПерекладу : переклад доріжки >
Транскрипт "1" -- "0..1" ПереказДоріжки : переказ >
ЗбереженеСлово "0..*" -- "0..1" СловниковаСтаття : стаття довідника >
ЗбереженеСлово "0..*" -- "0..1" КешПояснень : пояснення в контексті >
ОцінкаРівня "0..1" -- "0..*" ВідомеСлово : поповнює профіль >


' ====================================================================
' ЗАЛЕЖНІСТЬ (dependency)
' ====================================================================
Відео ..> ДжерелоСубтитрів : читає доріжку через >
ФабрикаДжерелСубтитрів ..> ДжерелоСубтитрів : <<create>>
ПарсерДокумента ..> Документ : <<create>>
ЗбереженеСлово ..> ВідомеСлово : <<create>>


' ====================================================================
' ПРИМІТКИ ДО КЛЮЧОВИХ РІШЕНЬ МОДЕЛІ
' ====================================================================
note top of "Спільні довідники без власника"
  Усі шість класів **навмисно без зв'язку
  з Користувачем**: ключ — вміст або
  ідентифікатор ролика, тому «хто дивився»
  з них не дізнатися. Спільними є саме
  розпізнавання й переклад доріжки.
  Докладно: Розділ 6.1.
end note


note right of Документ
  Стаття за посиланням — це Документ
  з форматом «стаття-html» (UC12), тому
  нащадків рівно два; окремий клас для
  вебсторінки — поза межами роботи
  (розділ «Мета»). Зв'язок «документ —
  рядок історії» виражений узагальненням,
  тому асоціації «Документ — Матеріал» немає.
end note
```

```
note bottom of ЛічильникВикористання
  Технічного id немає: рядок
  ідентифікується парою «день + вид»
  у межах власника — обидва атрибути
  позначено модифікатором {id}.
end note



@enduml
```

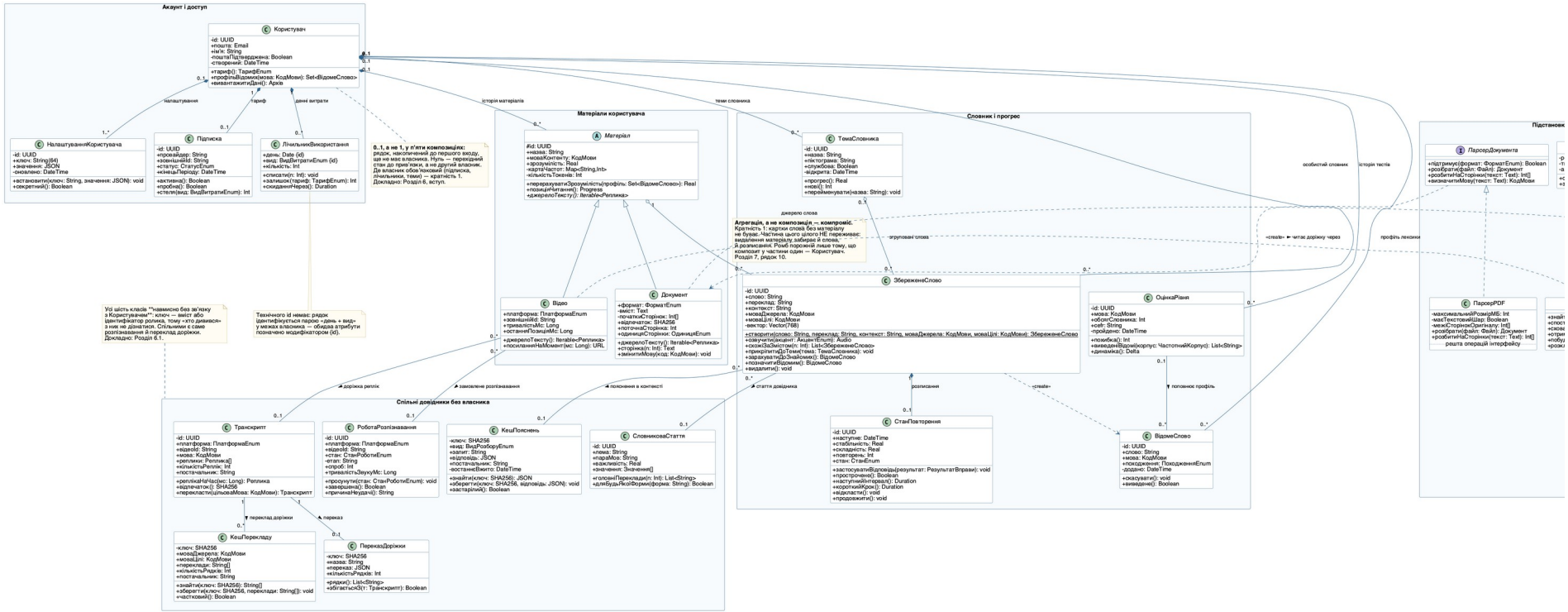


Рисунок 1 — Повна діаграма класів предметної області «Півот»

Висновок: повна діаграма містить 23 елементи і 26 зв’язків, серед них усі п’ять обов’язкових типів відношень; оскільки на аркуші звіту така діаграма читається погано, нижче наведено три часткові види тієї самої моделі.

Розділ 5. Три часткові види

Часткові види не є іншими діаграмами: у них не змінено жодного класу, жодного атрибута й жодного типу відношення — показано підмножини тієї самої моделі у збільшеному масштабі. Класи, наведені як контекст, подані скорочено.

Одне розходження треба назвати прямо, щоб вид не прочитався як інша модель. Види 1 і 2 — **точні підмножини** рисунка 1. Вид 3 є підмножиною **плюс чотири класи, яких на повній діаграмі немає зовсім**: `ДжерелоСубтитрівВласнийФайл`, `ДжерелоСубтитрівРозпізнане`, `ПарсерEPUB` і `ПарсерПростогоТексту`. Вони позначені сірим і наведені не як частина моделі, а як доказ підстановності двох контрактів: саме на них видно, що реалізацій може бути скільки завгодно й що нова не змінює ні предка, ні клієнта. Разом із ними на виді 3 з’являються ще чотири реалізації — тобто шість замість двох на рисунку 1. Усі 23 елементи самої моделі покриті білими класами трьох видів без жодного пропуску.

5.1. Вид 1 — акаунт і його частини

Перший вид показує, що саме належить акаунту безпосередньо: налаштування, тариф, денні лічильники, історію тестів і профіль відомої лексики. Усі п’ять зв’язків тут — композиції: жодна з цих частин не має сенсу без власника й видаляється разом із ним.

Тут же видно два рішення про кратність, які легко проґавити. Перше: з боку `Користувач` у трьох із п’яти композицій стоїть `0..1`, а не `1`, — у схемі рядки, накопичені до першого входу, лишаються без власника й дістаються першому зареєстрованому, тож нуль є перехідним станом до прив’язки. У підписки й денних лічильників власник обов’язковий, і там збережено `1`. Друге: шостий зв’язок, `ОцінкаРівня – ВідомеСлово`, навмисно показано **асоціацією** з кратністю `0..1 – 0..*`. Асоціацією — бо тест обсягу словника поповнює профіль лексики (це `<<include>>` UC17 → UC07 з ЛР1), але видалення запису тесту не повинне забирати знання користувача про слова. А `0..1`, а не `1`, — бо слово, позначене вручну в UC07, жодної оцінки рівня не має: це перше зі двох значень `ПоходженняEnum`, і воно ж є типовим.

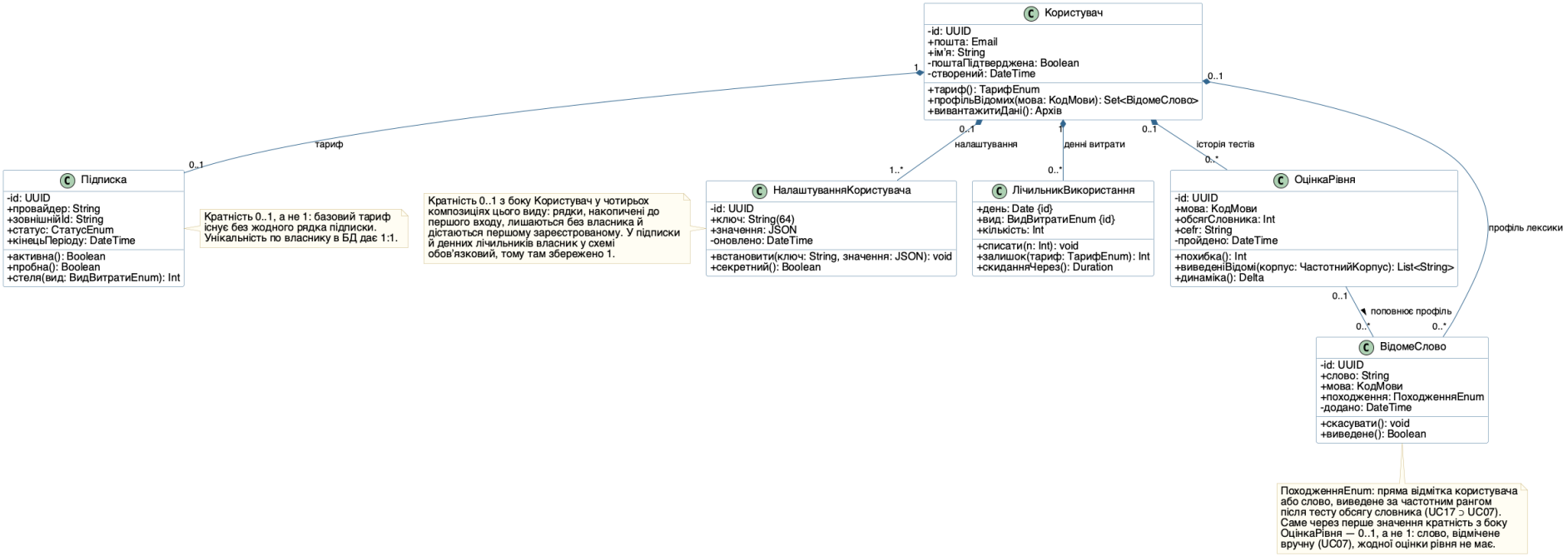


Рисунок 2 — Частковий вид 1: акаунт і його частини

5.2. Вид 2 — словник, повторення й спільний довідник

Другий вид показує найтоншу частину моделі: одне й те саме `ЗбереженеСлово` входить у три різні відношення «ціле-частина» одночасно. Від `Користувач` — композиція з кратністю `0..1` з боку власника (слово видаляється разом з акаунтом, але до першого входу може бути ще безвласним). Від `Матеріал` — агрегація з кратністю `1` (картка слова без матеріалу не створюється взагалі). Від `ТемаСловника` — агрегація з кратністю `0..1` (тема лише групує слова, і слово спокійно лежить без теми). Саме тут проходить головна межа між агрегацією й композицією, і саме тут ШІ-чернетка помилилася двічі (див. розділ «Критичний аналіз ШІ-чернетки»). Сьомий зв’язок виду — асоціація `ЗбереженеСлово – КешПояснень`: відповідь ШІ про слово в контексті спільна й без власника, тому повторний той самий запит не витрачає денної квоти (UC08, UC09).

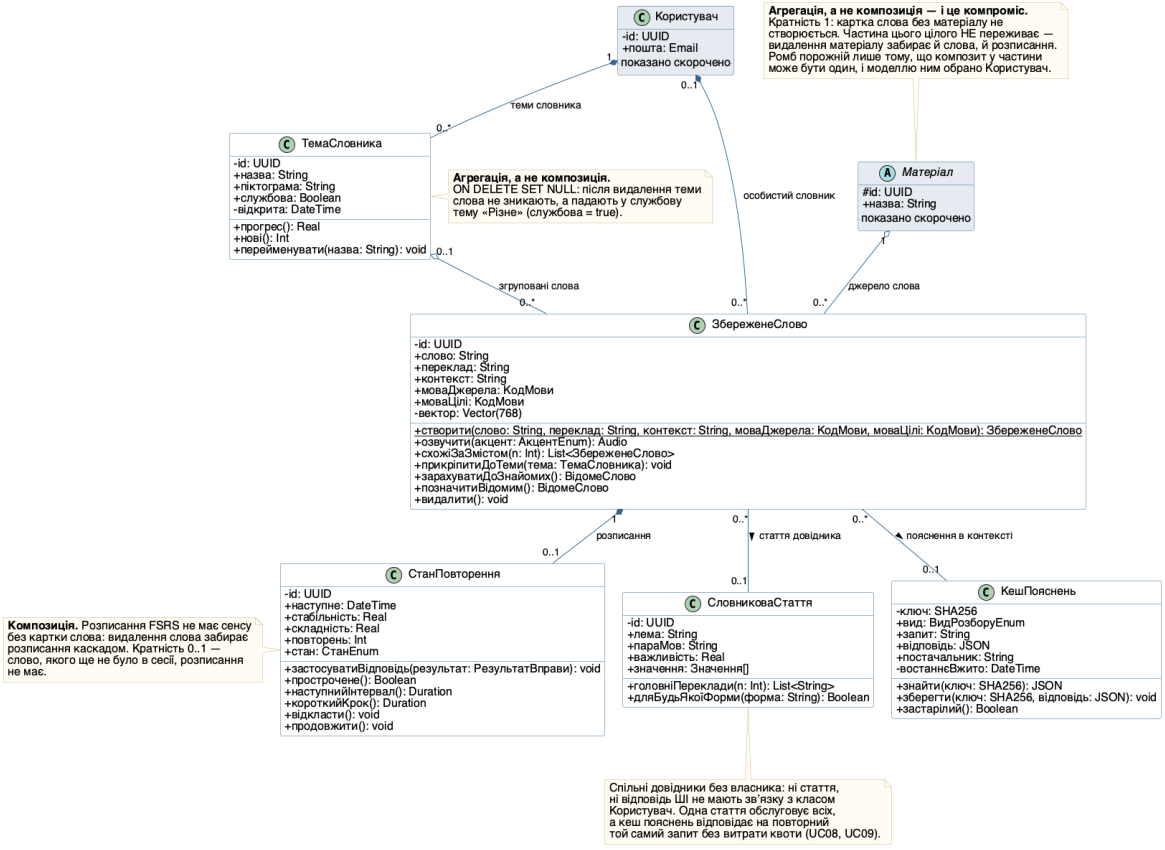


Рисунок 3 — Частковий вид 2: словник, повторення й спільний довідник

5.3. Вид 3 — матеріали, транскрипти й точки підстановки

Третій вид збирає разом обидва обов’язкові «ієрархічні» відношення: узагальнення (`Матеріал ◁– Відео`, `Матеріал ◁– Документ`) і реалізацію інтерфейсу. Реалізацій тут показано шість, а не дві, як на рисунку 1: чотири додаткові класи

позначено сірим і вони, як сказано на початку розділу, **не входять до моделі** — вони наведені лише щоб було видно, що контракт справді підстановний. Цей вид найкраще показує, чому реалізація інтерфейсу — це **не** узагальнення: `ДжерелоСубтитрівYouTube` не є різновидом чогось, він лише обіцяє виконувати контракт, і між трьома реалізаціями немає жодного спільного стану.

Ще одна відмінність виду 3 стосується не складу моделі, а масштабу. На рисунку 1 операції двох класів-реалізацій згорнуто розділювачем `.. решта операцій інтерфейсу ..`, бо на повній діаграмі вони не читаються: `ДжерелоСубтитрівYouTube` показує там три операції, `ПарсерPDF` — дві. На виді 3 той самий склад виписано повністю — сім і чотири рядки відповідно. Це розгортання згорнутого переліку, а не інші класи: жодної операції на виді 3 не додано й не прибрано.

Білою частиною виду, тобто самою моделлю, тут є ще два класи пакета спільних довідників: `РоботаРозпізнавання` (замовлення доріжки в сервісу ШІ з UC10 — стан, етап, кількість спроб) і `КешПерекладу` (переклад доріжки за відбитком «мови + рядки»). Обидва без власника, обидва ключовані вмістом або ідентифікатором ролика в площадки, і саме вони роблять твердження «другий глядач того самого ролика не платить» перевірним.

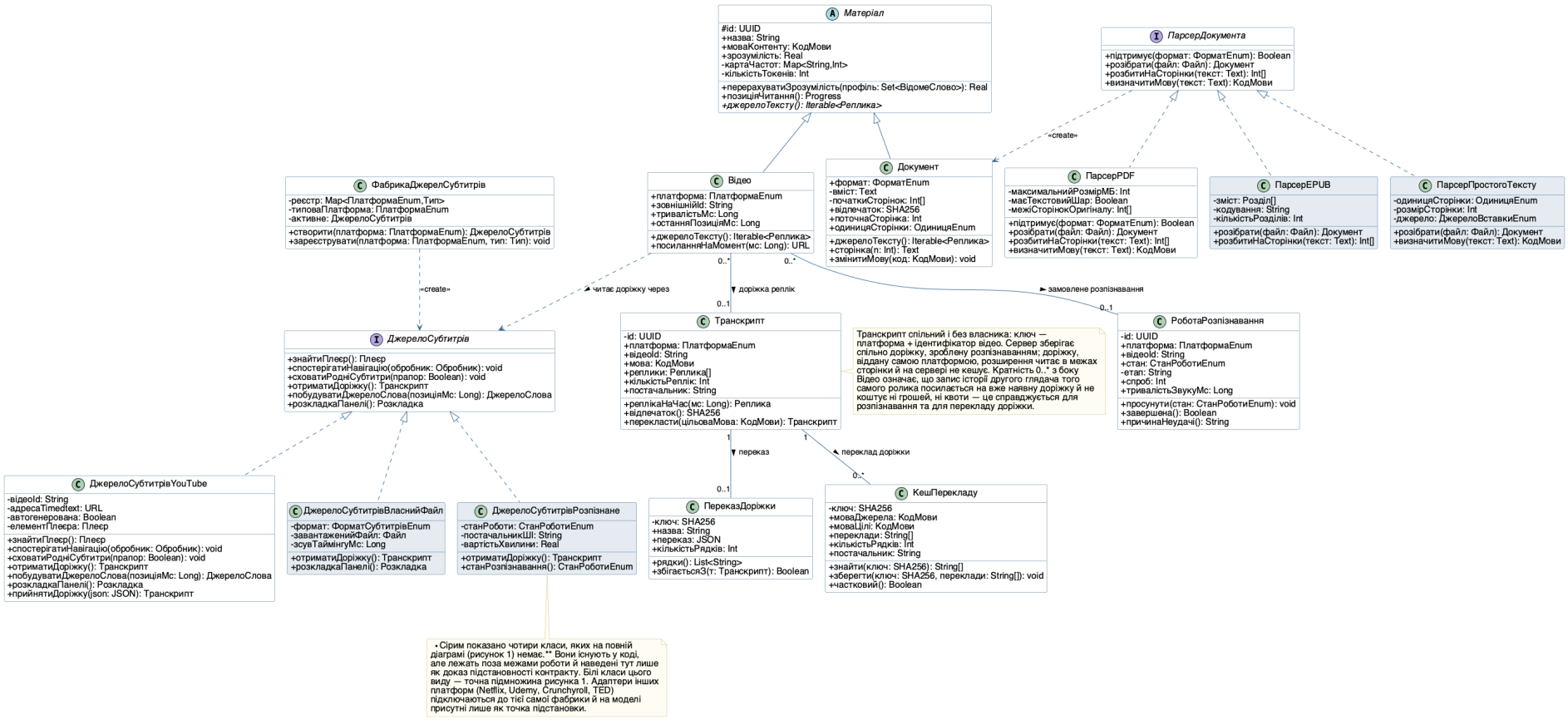


Рисунок 4 — Частковий вид 3: матеріали, транскрипти й точки підстановки

Висновок: три часткові види покривають усі 23 елементи й усі п’ять типів відношень повної діаграми; моделі вони не змінюють, а єдине додавання — чотири сірі класи реалізацій на виді 3 — названо прямо й винесене за межі моделі. У звіті види виконують роль збільшувального скла, а не альтернативи рисунку 1.

Розділ 6. Усі п’ять типів відношень і множинність

Нижче перелічено кожне відношення повної діаграми з його типом, множинністю й підставою. Множинність читається за правилом методички: `Користувач "1" *-- "0..*" ТемаСловника` означає, що один користувач має від нуля до багатьох тем, а кожна тема належить рівно одному користувачеві.

Окремо треба пояснити кратність `0..1` з боку `Користувач` у п’яти композиціях — вона виглядає як помилка, але нею не є. У схемі даних власник рядка **nullable** в налаштуваннях, історії матеріалів, збережених словах, профілі лексики й історії тестів: рядки, накопичені до появи входу, лишаються без власника й дістаються першому зареєстрованому. Нуль тут означає «ще не прив’язано», а не «другий власник», тому відношення лишається композицією: власник усе одно може бути лише один. Там, де власник у схемі обов’язковий — підписка, денні лічильники, теми словника, — збережено кратність `1`. Саме через це в Розділі 2.1 сказано «узгоджене зі схемою», а не «тотожне їй».

Відношення	Множинність	Тип	Підстава з ЛР1 / моделі
`Користувач` — `НалаштуванняКористувача`	0..1 — 1..*	Композиція	UC01: мастер першого запуску завжди створює щонайменше мовну пару, тому `1..*`, а не `0..*`; уточнюється в UC03. Нуль з боку власника — безвласні рядки до першого входу
`Користувач` — `Підписка`	1 — 0..1	Композиція	UC18: базовий тариф існує без жодного рядка підписки; унікальність по власнику дає не більше однієї. Власник у схемі обов’язковий, тому `1`
`Користувач` — `ЛічильникВикористання`	1 — 0..*	Композиція	UC04, UC05, UC12: один рядок на пару «день + вид витрати».

			Власник обов’язковий, тому `1`
`Користувач` — `Матеріал`	0..1 — 0..*	Композиція	UC04, UC12: історія матеріалів акаунта; нуль — рядок історії, накопичений до входу
`Користувач` — `ЗбереженеСлово`	0..1 — 0..*	Композиція	UC06, UC14: особистий словник; нуль — слово, збережене до входу
`Користувач` — `ТемаСловника`	1 — 0..*	Композиція	UC14, UC15: теми словника особисті, бо групують особисті слова. Власник обов’язковий, тому `1`
`Користувач` — `ВідомеСлово`	0..1 — 0..*	Композиція	UC07: профіль відомої лексики, окремий для кожної мови; нуль — відмітка до входу
`Користувач` — `ОцінкаРівня`	0..1 — 0..*	Композиція	UC17: історія зростання рівня; нуль — тест, пройдений до входу
`ЗбереженеСлово` — `СтанПовторення`	1 — 0..1	Композиція	UC16: слово, яке ще не потрапляло в сесію, розписання не має; посилання на слово обов’язкове
`Матеріал` — `ЗбереженеСлово`	1 — 0..*	Агрегація	UC06: картка слова завжди зберігається з матеріалу, і посилання на нього обов’язкове, тому `1`, а не `0..1`. Ромб порожній лише тому, що композит у частини може бути один, і моделлю ним обрано `Користувач` (див. Розділ 7, рядок 10)
`ТемаСловника` — `ЗбереженеСлово`	0..1 — 0..*	Агрегація	UC15: нерозібране слово лежить без теми, у «Різному»
`Відео` — `Транскрипт`	0..* — 0..1	Асоціація	UC04: доріжка спільна за ключем «платформа + ідентифікатор відео»
`Відео` —	0..* — 0..1	Асоціація	UC10: одна робота на ролик, спільна для всіх,

`РоботаРозпізнавання`			хто його відкрив; нуль — доріжка є й замовляти нічого
`Транскрипт` — `КешПерекладу`	1 — 0..*	Асоціація	UC04: на одну доріжку буває кілька перекладів — по одному на цільову мову й постачальника
`Транскрипт` — `ПереказДоріжки`	1 — 0..1	Асоціація	UC11: переказ спільний, ключ — відбиток рядків доріжки
`ЗбереженеСлово` — `СловниковаСтаття`	0..* — 0..1	Асоціація	UC05, крок 3: стаття довідника спільна на всіх користувачів
`ЗбереженеСлово` — `КешПояснень`	0..* — 0..1	Асоціація	UC08, UC09: відповідь ШІ спільна за відбитком запиту; нуль — пояснення ще не замовляли
`ОцінкаРівня` — `ВідомеСлово`	0..1 — 0..*	Асоціація	UC17 `<<include>>` UC07: тест поповнює профіль, але не володіє ним. `0..1`, а не `1`, бо слово з походженням «пряма відмітка користувача» (UC07) жодної оцінки рівня не має
`Матеріал` ◁— `Відео`	—	Узагальнення	Нащадок з’являється там, де відрізняється реалізація `джерелоТексту()`: у відео це доріжка реплік із часовими мітками
`Матеріал` ◁— `Документ`	—	Узагальнення	У документі `джерелоТексту()` дає сторінки, розбиті на речення. Стаття за посиланням тим самим методом не відрізняється, тому входить як `Документ` з форматом «стаття-html», а не як третій нащадок (межі роботи,

			розділ «Мета»)
`ДжерелоСубтитрів` ◁┈┈ `ДжерелоСубтитрівYouTube`	—	Реалізація	UC04, крок 3: звернення до сервісу субтитрів YouTube
`ПарсерДокумента` ◁┈┈ `ПарсерPDF`	—	Реалізація	UC12, кроки 4–5: вибір парсера за форматом
`Відео` ┈┈> `ДжерелоСубтитрів`	—	Залежність	Відео читає доріжку лише через інтерфейс, не знаючи реалізації
`ФабрикаДжерелСубтитрів` ┈┈> `ДжерелоСубтитрів`	—	Залежність `<<create>>`	Підстановка реалізації за платформою
`ПарсерДокумента` ┈┈> `Документ`	—	Залежність `<<create>>`	Документ створюється парсером відповідного формату
`ЗбереженеСлово` ┈┈> `ВідомеСлово`	—	Залежність `<<create>>`	UC07 і UC16: операції `зарахуватиДоЗнайомих()` і `позначитиВідомим()` створюють запис профілю лексики й повертають його (Розділ 9.1). Саме залежність, а не відношення «ціле-частина»: `ВідомеСлово` належить композицією класові `Користувач` і переживає картку, що його створила

6.1. Чому шість класів навмисно не пов’язані з `Користувач`

Увесь пакет «Спільні довідники без власника» — `Транскрипт`, `РоботаРозпізнавання`, `КешПерекладу`, `КешПояснень`, `ПереказДоріжки` і `СловниковаСтаття` — не має жодної асоціації з класом `Користувач`. Це не пропуск, а рішення, прийняте одночасно з міркувань приватності й економії.

Приватність. Ці класи ключуються вмістом або ідентифікатором ролика в площадки: доріжка — парою «платформа + ідентифікатор відео», переклад доріжки — відбитком «мови + рядки», пояснення — відбитком «слово + контекст + мови», переказ — відбитком рядків доріжки, стаття — лемою й парою мов. Поля «хто це дивився» в записі немає й не може бути. Отже, навіть маючи повний

доступ до цих таблиць, неможливо відновити, що саме конкретна людина дивилася чи читала. Рядок каже лише, що ролик хтось розшифровував, — і не каже хто.

Економія. Кратність `0..*` з боку `Відео` в асоціаціях з `Транскрипт` і `РоботаРозпізнавання` означає буквально таке: другий глядач того самого ролика посилається на вже наявну доріжку й на вже виконану роботу. Він не витрачає ні грошей на зовнішній сервіс, ні власної денної квоти — що прямо описано постумовою UC04: «переклад доріжки збережений у спільному кеші й доступний наступним глядачам того самого відео без повторних витрат». Тут важливо бути точним у межах цього твердження: спільними є саме **розпізнавання** й **переклад**. Доріжку, яку віддає сама платформа, розширення читає в межах сторінки й на сервері не зберігає — тому для відео з готовими субтитрами економія стосується перекладу, а не самої доріжки.

Чому в пакеті саме шість класів. У схемі однотипних спільних кешів більше: окремо лежать кеш перекладу, кеш пояснень слова, кеш розборів речення, черга робіт розпізнавання, частотний кеш і журнал списань за рядки перекладу. У модель внесено шість, і вибір зроблено за правилом «новий клас має або дати варіанту використання власну сутність, або додати новий тип відношення». Саме тому пояснення слова й розбір речення зведені в один `КешПояснень` з атрибутом `вид: ВидРозборуEnum`: таблиці різні, бо різні ключі й промпти, але на рівні предметної області це той самий факт — «відповідь ШІ, яку система пам’ятає». Частотний кеш і журнал списань нових сутностей варіантам використання не дають і нових відношень не вносять, тому лишилися поза моделлю.

Висновок: на діаграмі представлено всі п’ять обов’язкових типів відношень, у кожному відношенні «ціле-частина» й у кожній асоціації явно зазначено множинність обох кінців, а шість класів-довідників навмисно лишені без власника — і це рішення, а не недогляд.

Розділ 7. Обґрунтування агрегації й композиції за тестом розділу 1.2.5

Методичка вимагає письмово обґрунтувати вибір для **кожного** відношення «ціле-частина». На діаграмі їх одинадцять: дев’ять композицій і дві агрегації. До кожного застосовано той самий тест: «Якщо видалити ціле, чи має сенс частині продовжувати існувати сама по собі?» — і додатково другий критерій із порівняльної таблиці методички: «Чи може частина належати кільком цілим одночасно?»

Один випадок із одинадцяти розходиться з першим критерієм, і це розходження в таблиці нижче не згладжено, а названо: у рядку 10 частина цілого **не** переживає, але позначено агрегацію — бо інакше той самий клас отримав би два суцільні ромби. Решта десять відповідей на тест прямі.

Кратність `0..1` з боку `Користувач` у п’яти композиціях відповідей на тест не змінює. Тест питає, що станеться з частиною при видаленні цілого, — і відповідь усюди та сама: частина видаляється каскадом. Нуль описує інший момент життя рядка — проміжок до першого входу, коли власника ще немає; щойно він з’являється, рядок прив’язується назавжди й іншого власника отримати не може. Саме неможливість другого власника, а не обов’язковість першого, і робить відношення композицією.

№	Відношення	Обраний тип	Відповідь на тест «чи має сенс частині існувати без цілого»
1	`Користувач` ◆ `НалаштуванняКористувача`	Композиція	Ні. Рядок «мовна пара = en→uk» поза конкретним акаунтом не означає нічого: це не довідник налаштувань, а значення саме цієї людини. Видалення акаунта забирає всі рядки
2	`Користувач` ◆ `Підписка`	Композиція	Ні. Підписка — це стан акаунта в платіжного провайдера; без власника вона не має ні предмета, ні платника. При видаленні акаунта запис підписки видаляється разом із ним
3	`Користувач` ◆ `ЛічильникВикористання`	Композиція	Ні. Запис «15 перекладів за 3 жовтня» без власника нічого не обмежує й нічого не описує — це не статистика сервісу, а особистий ліміт
4	`Користувач` ◆ `Матеріал`	Композиція	Ні. Запис історії — це «мій перегляд цього відео» з моєю позицією

			й моєю зрозумілістю. Саме відео на платформі далі існує, але запис історії без власника змісту не має
5	`Користувач` ◆ `ЗбереженеСлово`	Композиція	Ні. Картка слова містить мій контекст, мій переклад і моє розписання повторень; це особисті дані, що видаляються разом з акаунтом
6	`Користувач` ◆ `ТемаСловника`	Композиція	Ні. Тема групує мої слова: «Кулінарія» в моєму словнику й у чужому — це два різні набори. Без власника тема порожня за означенням
7	`Користувач` ◆ `ВідомеСлово`	Композиція	Ні. Профіль відомої лексики — це твердження «я знаю це слово»; поза акаунтом воно не є ні фактом про мову, ні фактом про когось
8	`Користувач` ◆ `ОцінкаРівня`	Композиція	Ні. Результат «≈4200 слів, B1» без того, чий він, — просто число. Історія тестів належить акаунту
9	`ЗбереженеСлово` ◆ `СтанПовторення`	Композиція	Ні. Розписання FSRS (наступна дата, стабільність, складність) описує саме цю картку. Видалення слова зі словника забирає розписання каскадом; «безхазяйний» стан повторення нічого не планує
10	`Матеріал` ◇ `ЗбереженеСлово`	Агрегація (свідомий компроміс)	Ні — і це треба сказати прямо. Частина цього цілого не переживає: видалення матеріалу з

			історії видаляє і його слова, і їхні розписання повторень, причому безумовно. Посилання на матеріал у схемі обов’язкове, звідси й кратність `1`. За першим критерієм методички це була б композиція. Агрегацію обрано виключно через другий критерій: частина може мати лише один композит, і моделлю ним обрано `Користувач` (рядок 5) — на рівні предметної області картка слова є перш за все особистими даними. Два суцільні ромби на одному класі зробили б модель суперечливою, тому з боку матеріалу ромб порожній, а фактичний каскад зафіксовано приміткою просто на зв’язку. Це єдине місце моделі, де тест методички й обране позначення розходяться, і розходження лишено видимим, а не прихованим
11	`ТемаСловника` ◇ `ЗбереженеСлово`	Агрегація	Так, і це перевірено поведінкою системи. При видаленні теми слова не зникають, а падають у службову тему «Різне» — у БД це `ON DELETE SET NULL`. Крім того, слово існує без теми взагалі: саме нерозібрані слова є

			входом для UC15 «Розкласти словник за темами». Отже, частина переживає ціле — агрегація

7.1. Чому жодне відношення не лишилося простою асоціацією «за замовчуванням»

Приклад методички окремо показує випадок `Клієнт – Замовлення`, де ромба не має бути взагалі: замовлення не є структурною частиною клієнта. У цій моделі такий самий статус мають усі сім асоціацій з Розділу 6. `ЗбереженеСлово – СловниковаСтаття` — найчистіший приклад: стаття довідника не є частиною картки слова, вона спільна для всіх і живе своїм життям; картка лише посилається на неї. Тут доречно уточнити два альтернативні потоки UC05, бо вони описують різні випадки: коли **прямої пари мов у довіднику немає**, стаття будується через англійську й у картці зазначається, що переклад непрямий (потік 3а), а коли **недоступний сам сервіс перекладу**, картка показує лише словникову статтю або повідомлення «переклад тимчасово недоступний» (потік 4а). Ні той, ні інший випадок не робить статтю частиною картки. Так само `Відео – Транскрипт`: доріжка не є частиною відео в акаунті, а спільним ресурсом, на який посилаються записи історії різних людей.

Висновок: усі одинадцять відношень «ціле-частина» обґрунтовано тестом методички. `ТемаСловника ◇ ЗбереженеСлово` — чиста агрегація: частина переживає ціле, у схемі це `ON DELETE SET NULL`, а слова падають у «Різне». `Матеріал ◇ ЗбереженеСлово` — агрегація за другим критерієм, а не за першим: частина не переживає, але композит у неї вже є, і це єдине свідомо лишене розходження моделі з тестом.

Розділ 8. Перевірка узгодженості з Лабораторною роботою №1

Трасованість перевірено на трьох рівнях: актори ЛР1 → елементи моделі класів, варіанти використання → задіяні класи й, найсуворіше, ключові іменники текстових специфікацій → класи або атрибути.

8.1. Актори ЛР1 на діаграмі класів

Головне, що треба зафіксувати: **актор і клас — різні речі**. Актор — це роль у взаємодії, клас — сутність із власним станом. Тому відображення не є взаємно однозначним, і кожне розходження має пояснення.

Актор ЛР1	Що йому відповідає в моделі	Пояснення

	класів	
Користувач сервісу *(абстрактний)*	Не має власного класу	Його єдина спільна властивість — мовна пара й мова інтерфейсу — зберігається рядками `НалаштуванняКористувача`; абстрактним класом у цій моделі став `Матеріал`, а не користувач
Гість	Не має класу	До виконання UC01 у системі немає жодного запису; гість — це відсутність рядка `Користувач`
Зареєстрований користувач	`Користувач`	Пряме відображення: пошта, ім’я, підтвердження пошти, дата створення
Користувач з підпискою	Не окремий клас, а стан	`Користувач.тариф()` обчислюється з `Підписка.активна()`. Окремий клас-нащадок створив би два види користувачів у базі, хоча різниця між ними — один рядок у таблиці підписок
Адміністратор	Не має класу	Роль обслуговування, що діє поверх тих самих сутностей (`Підписка`, `ЛічильникВикористання`); окремих даних про себе система не зберігає
Зовнішній постачальник ШІ-функцій *(абстрактний)*	Не має класу	Зовнішня система не є сутністю нашої предметної області; у ЛР3 вона стане компонентом «Шар зовнішніх сервісів»
Сервіс ШІ	Сліди роботи, не клас	`РоботаРозпізнавання`, `КешПояснень`, `Транскрипт.постачальник`, `ЗбереженеСлово.вектор`, `ПереказДоріжки`, `ТемаСловника`
Сервіс перекладу	Сліди роботи, не клас	`КешПерекладу`, `ЗбереженеСлово.переклад`, `Транскрипт.перекласти()`, `ЛічильникВикористання` з видом «переклад»
Сервіс субтитрів YouTube	`ДжерелоСубтитрівYouTube`	Єдиний актор, що отримав на діаграмі класів прямий відповідник — саме тому, що звернення до нього

		інкапсульоване інтерфейсом
Платіжний провайдер	Поля, не клас	`Підписка.провайдер`, `ПідписказовнішнійId`, `Підписка.статус`
Провайдер авторизації	Не має класу	Запис зовнішньої автентифікації належить шару доступу (компонент «API сервісу та контроль доступу» в ЛР3), а не предметній області
Поштовий сервіс	Один прапорець	`Користувач.поштаПідтверджена`; самі листи система не зберігає

8.2. Трасування «Use Case → задіяні класи»

Use Case ЛР1	**Задіяні класи, атрибути й методи**
UC01 Зареєструватися в сервісі	`Користувач` (`пошта`, `ім’я`, `створений`), `НалаштуванняКористувача.встановити()` — мовна пара й акцент у мастері першого запуску
UC02 Увійти в сервіс	`Користувач` (`пошта`, `поштаПідтверджена`)
UC02.1 Увійти поштою або паролем	`Користувач.поштаПідтверджена`
UC02.2 Увійти через зовнішній акаунт	`Користувач.пошта`
UC03 Налаштувати параметри вивчення	`НалаштуванняКористувача` (`ключ`, `значення`, `встановити()`, `секретний()` — для власного ключа ШІ)
UC04 Дивитися відео з двомовними субтитрами	`Відео`, `Транскрипт` (`реплікаНаЧас()`, `перекласти()`, `відпечаток()`), `КешПерекладу` (`знайти()`, `частковий()`), `ДжерелоСубтитрів`, `ДжерелоСубтитрівYouTube`, `ФабрикаДжерелСубтитрів`, `Матеріал.перерахуватиЗрозумілість()`, `ЛічильникВикористання.списати()`
UC05 Отримати значення слова	`СловниковаСтаття` (`головніПереклади()`, `дляБудьЯкоїФорми()`), `КешПерекладу` — коли прямої статті немає, `Транскрипт.реплікаНаЧас()`, `ЗбереженеСлово.озвучити()`, `ЛічильникВикористання`
UC06 Зберегти слово у словник	`ЗбереженеСлово` (`слово`, `переклад`, `контекст`, `моваДжерела`, `моваЦілі`), агрегація від `Матеріал`
UC07 Позначити слово як відоме	`ВідомеСлово` (`походження`, `скасувати()`), `Користувач.профільВідомих()`
UC08 Отримати пояснення слова в контексті	`КешПоясснень` (`вид` = пояснення слова, `знайти()`, `застарілий()`), `ЗбереженеСлово.контекст`,

	`ЛічильникВикористання.залишок()`, `Підписка.стеля()`
UC09 Розібрати будову речення	`КешПоясснень` (`вид` = розбір речення, `відповідь`), `Транскрипт.реплікаНаЧас()`, `Документ.сторінка()`, `Підписка.активна()`
UC10 Замовити розпізнавання мовлення для відео	`РоботаРозпізнавання` (`стан`, `спроб`, `просунути()`, `завершена()`, `причинаНеудачі()`), `Транскрипт.постачальник`, `Підписка.активна()`
UC11 Отримати переказ матеріалу	`ПереказДоріжки` (`переказ`, `рядки()`, `збігаєтьсяЗ()`), `Матеріал.зрозумілість`, `перерахуватиЗрозумілість()`, `Підписка.активна()`, `Підписка.стеля(вид)`
UC12 Завантажити власний матеріал	`Документ` (`формат`, `вміст`, `відпечаток`, `змінитиМову()`), `ПарсерДокумента`, `ПарсерPDF`, `ПарсерEPUB`, `ПарсерПростогоТексту`, `Матеріал.картаЧастот`
UC13 Читати власний текст у читалці	`Документ` (`сторінка()`, `поточнаСторінка`, `початкиСторінок`, `одиницяСторінки`), `Матеріал.позиціяЧитання()`
UC14 Працювати зі словником за темами	`ТемаСловника` (`прогрес()`, `нові()`, `перейменувати()`), `ЗбереженеСлово`, `СтанПовторення.наступне`, `Користувач.вивантажитиДані()`
UC15 Розкласти словник за темами	`ЗбереженеСлово` (`вектор`, `схожіЗаЗмістом()`, `прикріпитиДоТеми()`), `ТемаСловника.службова` — тема «Різне», `Підписка.активна()`, `Підписка.стеля(вид)`
UC16 Пройти сесію інтервальних повторень	`СтанПовторення` (`застосуватиВідповідь()`, `прострочене()`, `наступнийІнтервал()`, `стан`), `ЗбереженеСлово`
UC17 Пройти тест обсягу словника	`ОцінкаРівня` (`обсягСловника`, `cefr`, `похибка()`, `виведеніВідомі()`, `динаміка()`), `ВідомеСлово.походження`
UC18 Оформити й керувати підпискою	`Підписка` (`статус`, `кінецьПеріоду`, `активна()`, `пробна()`, `стеля()`), `ЛічильникВикористання.залишок()`

Жоден із 18 варіантів використання не лишився без класів, і жоден клас не лишився без варіанта використання — перевірка пройдена в обидва боки. Усі класи, названі в таблиці, присутні на повній діаграмі (рисунок 1): жоден варіант використання не трасується на клас, якого в моделі немає. Окремо звірено гейт підписки: усі чотири платні варіанти використання, які в ЛР1 належать акторові

«Користувач з підпискою» (UC09, UC10, UC11, UC15), трасуються на `Підписка.активна()` або `Підписка.стеля(вид)` однаково.

8.3. Звіряння ключових іменників текстових специфікацій

Найсуворіша перевірка методички: кожен ключовий іменник із повних текстових специфікацій UC04, UC05 і UC12 має бути присутній серед класів або атрибутів. Нижче — повний перелік; іменники, що описували дію або екранний елемент, позначено окремо.

Іменник зі специфікації ЛР1	**Де присутній у моделі класів**
плеєр	`ДжерелоСубтитрів.знайтиПлеєр()`, `ДжерелоСубтитрівYouTube.елементПлеєра`
ідентифікатор відео	`Відео.зовнішнійId`, `Транскрипт.відеоId`
доріжка реплік	Клас `Транскрипт`
репліка	Тип значення `Реплика` в `Транскрипт.реплики` і `джерелоТексту()`
мова оригіналу	`Матеріал.моваКонтенту`, `Транскрипт.мова`, `ЗбереженеСлово.моваДжерела`
переклад доріжки	`Транскрипт.перекласти(цільоваМова)`
спільний кеш	Класи `Транскрипт`, `КешПерекладу`, `КешПоясснень`, `ПереказДоріжки` без власника (Розділ 6.1)
відбиток	`Транскрипт.відпечаток()`, `Документ.відпечаток`, `ПереказДоріжки.ключ`
денний ліміт	Клас `ЛічильникВикористання`, метод `залишок(тариф)`
профіль відомої лексики	Клас `ВідомеСлово`, метод `Користувач.профільВідомих(мова)`
слово	`ЗбереженеСлово.слово`, `ВідомеСлово.слово`
підсвічування незнайомих слів	Похідне від `профільВідомих()`; стану не має, тому не клас
режим показу субтитрів	Рядок `НалаштуванняКористувача`
панель транскрипту	`ДжерелоСубтитрів.розкладкаПанелі()`
позиція перегляду	`Відео.останняПозиціяМс`, `Матеріал.позиціяЧитання()`
зрозумілість матеріалу	`Матеріал.зрозумілість`, `перерахуватиЗрозумілість(профіль)`
історія матеріалів	Композиція `Користувач ◆— Матеріал` з кратністю `0..*`
тариф	`Користувач.тариф()`, клас `Підписка`
розпізнавання мовлення	Клас `РоботаРозпізнавання` (`стан: СтанРоботиEnum`); браузерний бік — `ДжерелоСубтитрівРозпізнане` на частковому виді 3

джерело перекладу	`Транскрипт.постачальник`, `КешПерекладу.постачальник`
речення	`Транскрипт.реплікаНаЧас()`, `Документ.сторінка()`
лема	`СловниковаСтаття.лема`
словникова стаття	Клас `СловниковаСтаття`
пара мов	`СловниковаСтаття.параМов`, `ЗбереженеСлово.моваДжерела` + `моваЦілі`
значення й частини мови	`СловниковаСтаття.значення: Значення[]`
машинний переклад	`ЗбереженеСлово.переклад`
картка слова	Екранне подання; збирається з `СловниковаСтаття` і `ЗбереженеСлово` (Розділ 2.3)
контекстна репліка	`ЗбереженеСлово.контекст`
озвучка й акцент	`ЗбереженеСлово.озвучити(акцент: АкцентEnum)`
парсер	Інтерфейс `ПарсерДокумента` та його реалізації
формат, розширення, MIME-тип	`Документ.формат: ФорматEnum`, `ПарсерДокумента.підтримує(формат)`
межі сторінок оригіналу	`Документ.початкиСторінок`, `ПарсерPDF.межіСторінокОригіналу`
одиниця сторінки	`Документ.одиницяСторінки: ОдиницяEnum`
карта частот слів	`Матеріал.картаЧастот: Map<String,Int>`
мова тексту, визначена евристикою	`ПарсерДокумента.визначитиМову()`, `Документ.змінитиМову()`
обмеження розміру файла	`ПарсерPDF.максимальнийРозмірМБ`
запис в історії матеріалів	Екземпляр `Відео` або `Документ` як нащадків `Матеріал`
дублікат за відбитком вмісту	`Документ.відпечаток: SHA256`
читалка	Методи `Документ.сторінка(n)`, `поточнаСторінка`

Жоден ключовий іменник не лишився без місця. Три позиції — «підсвічування», «картка слова» і «читалка» — присутні як поведінка, а не як дані, і це зафіксовано явно: вони описують подання, а не стан, який система пам’ятає.

8.4. Пакети діаграми класів і компоненти Лабораторної роботи №3

Пакети на діаграмі класів — засіб читабельності, а не компоненти. Щоб ця теза не виглядала відмовкою, нижче наведено фактичну відповідність до восьми компонентів ЛР3. Вона **неповна й не взаємно однозначна**: пакетів п’ять, компонентів вісім, і один пакет розходиться на два компоненти.

Пакет діаграми класів (ЛР2)	Компонент ЛР3	Характер відповідності
Акаунт і доступ	4. API сервісу та контроль	Розходиться на два:

	доступу + 5. Тарифи й ліміти	`Користувач` і `НалаштуванняКористувача` належать четвертому, `Підписка` й `ЛічильникВикористання` — п’ятому
Матеріали користувача	7. Модуль контенту	Пряма відповідність
Словник і прогрес	6. Модуль навчання	Пряма відповідність
Спільні довідники без власника	7. Модуль контенту (кеші) + 8. Шар зовнішніх сервісів (постачальники)	Розходиться: самі записи зберігає модуль контенту, а наповнює їх шар зовнішніх сервісів
Підстановка джерел і парсерів	1. Оверлей розширення + 2. Адаптер платформи YouTube (інтерфейс `ДжерелоСубтитрів`); 7. Модуль контенту (інтерфейс `ПарсерДокумента`)	Єдиний пакет, що перетинає межу «браузер — сервер»
— (класу не відповідає)	3. Веб-інтерфейс	Компонент подання: власних даних не має, тому класу предметної області йому не відповідає

Висновок: узгодженість із ЛР1 перевірено в трьох зрізах; усі 18 варіантів використання мають класи, усі ключові іменники специфікацій знайшли місце серед класів, атрибутів або методів, а кожне розходження між актором і класом пояснено окремо — актор є роллю, а не сутністю. Відповідність «пакет класів → компонент ЛР3» існує, але частково: саме тому пакети не названо попереднім поділом на компоненти.

Розділ 9. Узгодження моделі з Лабораторними роботами №5 і №6

Діаграма станів (ЛР5) і комплект діаграм послідовності (ЛР6) будувалися **після** цієї роботи, і кожна з них закінчувалася власною перевіркою узгодженості з діаграмою класів. Разом вони дали **61 позицію**, адресовану діаграмі класів: шість правок у розділі 7.2 звіту ЛР5, п’ятдесят чотири повідомлення в розділі 8.4 звіту ЛР6 і ще одну позицію з таблиці розбіжностей 9.3 ЛР6 — метод `короткийКрок()`, якого в тих п’ятдесяти чотирьох не було. Решта рядків таблиці 9.3 ЛР6 або стосується правок самої діаграми станів ЛР5, або повторює позиції, уже враховані в розділі 8.4. Цей розділ розкладає весь перелік на дві нерівні частини й показує, що з ним зроблено.

Межа поділу — та сама, що її провів Розділ 2.3 цієї роботи: **доменна модель проти моделі проєктування**. Діаграма класів ЛР2 описує сутності предметної області, тобто те, що система пам’ятає між сеансами, і навмисно не містить ані керівних (control), ані межових (boundary) класів. Тому частина позицій є справжніми прогалинами вже затверджених класів — їх додано; а частина лише вказує на класи рівня проєктування, яких у доменній моделі бути не повинно, — їх свідомо не додано.

9.1. Доповнення за результатами ЛР5 і ЛР6

До діаграми додано **десять операцій і дві позиції щодо перелічуваних типів**. Жодного нового класу й жодного нового атрибута не створено: усі десять операцій вкладено у вже наявні класи, а дві позиції уточнюють таблицю розділу 3.2.

Одне нове відношення все ж з’явилося, і воно є прямим наслідком двох із цих операцій. `ЗбереженеСлово.зарахуватиДоЗнайомих()` і `ЗбереженеСлово.позначитиВідомим()` створюють і повертають об’єкт класу `ВідомеСлово` — саме так ЛР6 і показує їх, повідомленням «`create»(слово, мова, походження = виведене)` у групі 4 розділу 8.4. У цій моделі створення об’єкта позначається залежністю зі стереотипом `<<create>>`: так уже позначені `ФабрикаДжерелСубтитрів ..> ДжерелоСубтитрів` і `ПарсерДокумента ..> Документ`. Лишити дві операції з типом результату, до якого в їхнього класу немає на діаграмі жодного зв’язку, означало б зламати власну конвенцію, тому до `classes.puml` додано `ЗбереженеСлово ..> ВідомеСлово : <<create>>`. Залежностей стало чотири замість трьох, зв’язків — 26 замість 25. Відношень «ціле-частина» це не додає: `ВідомеСлово` належить композицією класові `Користувач` і переживає ту картку, що створила запис, — тому саме залежність, а не агрегація.

Метод або значення	До якого класу додано	Яка робота його виявила	Навіщо він потрібен
`+ {static} створити(слово: String, переклад: String, контекст: String, моваДжерела: КодМови, моваЦілі: КодМови): ЗбереженеСлово`	`ЗбереженеСлово`	ЛР6, група 4 розділу 8.4 — повідомлення `«create»` у під-взаємодії UC05 + UC06	Клас був, а операції народження картки не було. Діаграма послідовності показала, що картка створюється одним повідомленням із повним набором полів і що саме цей момент назавжди фіксує мовну пару «моваДжерела →

			моваЦілі»
`+ зарахуватиДоЗнайомих(): ВідомеСлово`	`ЗбереженеСлово`	ЛР6, група 4 розділу 8.4 (розбіжність 4 розділу 9.3)	Картка, що доросла до стану «Повторення», поповнює профіль лексики записом із походженням «виведене за частотним рангом», але сама продовжує жити й братиме участь у наступних сесіях
`+ позначитиВідомим(): ВідомеСлово`	`ЗбереженеСлово`	ЛР5, перехід 14; ЛР6, розбіжність 4 розділу 9.3	Пряма відмітка користувача (UC07). На відміну від попередньої операції, вона завершує життєвий цикл: у ЛР5 цей перехід веде в кінцевий псевдостан. У ЛР2 була лише зворотна операція `ВідомеСлово.скасувати()` — модель уміла скасувати відмітку, якої не вміла поставити
`+ видалити(): void`	`ЗбереженеСлово`	ЛР5, перехід 15	Каскад «видалення слова забирає розписання» був описаний лише текстом примітки до композиції, а операції, що цей каскад запускає, не оголошував жоден клас моделі
`+ короткийКрок(): Duration`	`СтанПовторення`	ЛР6, розбіжність 3 розділу 9.3	Для фаз «Вчиться» і «Перевчається» новий строк задається коротким кроком, а не повним інтервалом. Наявний `наступнийІнтервал()` обслуговує лише стан «Повторення», тому без другого методу короткі кроки фази навчання лишалися неявними
`+ відкласти(): void`	`СтанПовторення`	ЛР5, перехід 7; ЛР6,	Вивести картку з черги

		група 4 розділу 8.4	поточної сесії, запам’ятавши підстан фази навчання. Саме ця подія наповнює змістом псевдостан історії в ЛР5 — без неї історія стояла б «про всяк випадок»
`+ продовжити(): void`	`СтанПовторення`	ЛР5, перехід 8	Повернути відкладену картку в сесію. За істинної умови `прострочене()` вона повертається не в перший підстан фази навчання, а в той, де її спинили, — через псевдостан історії
`+ зберегти(ключ: SHA256, переклади: String[]): void`	`КешПерекладу`	ЛР6, група 4 розділу 8.4 — сценарій А	Клас умів лише `знайти()` і `частковий()`, тобто читати кеш, який ніхто не наповнює. Без операції запису твердження «другий глядач того самого ролика не платить ні грошима, ні квотою» не було перевірним на самій моделі
`+ зберегти(ключ: SHA256, відповідь: JSON): void`	`КешПояснень`	ЛР6, група 4 розділу 8.4 — сценарій В	Та сама прогалина в другому кеші: ЛР6 показала повідомлення запису відповіді ШІ відразу після її отримання від зовнішнього сервісу
`+ прийнятиДоріжку(json: JSON): Транскрипт`	`ДжерелоСубтитрівYouTube`	ЛР6, група 4 розділу 8.4 — сценарій А	Обробник асинхронної відповіді `timedtext (json3)`. Адаптер мав атрибут `адресаTimedtext: URL` і операцію `отриматиДоріжку()`, але не мав операції, яка перетворює сиру відповідь платформи в

			`Транскрипт`. Оголошено лише на реалізації, а не на інтерфейсі `ДжерелоСубтитрів`: формат json3 специфічний для YouTube, і контракт не повинен про нього знати
Значення `відкладена` в `СтанEnum`	атрибут `СтанПовторення.стан`, таблиця розділу 3.2	ЛР5, стан «Відкладена»	Атрибут мусить уміти зберігати п’ятий підстан життєвого циклу. Без цього значення діаграма станів описувала б стан, якого модель даних не має, — а це вже суперечність, а не доповнення
Значення `успішний` і `невдалий` перелічуваного типу `РезультатВправи`	параметр `СтанПовторення.застосуватиВідповідь()`, таблиця розділу 3.2	ЛР5, охоронні умови переходів 3–6 і 10–13; ЛР6, фрагмент `alt` під-взаємодії «Застосувати відповідь»	Тип фігурував у ЛР2 лише як тип параметра, і склад його значень не був розкритий. На ці значення посилаються вісім охоронних умов ЛР5 — по чотири на кожне значення, переходи 3–6 і 10–13 — і обидва операнди `alt` у ЛР6, тому без розкриття типу охоронним умовам не було на що посилатися. Дев’ятий перехід події `застосуватиВідповідь()` (`Нова → Вчиться`) охоронної умови не має свідомо: перша відповідь будь-якої якості виводить картку зі стану «нова»

Дві позиції цієї таблиці заслуговують окремої ремарки, бо виявилися **двічі, двома незалежними шляхами**. Операцію `СтанПовторення.відкласти()` ЛР5 вивела з

події переходу 7 діаграми станів, а ЛР6 — з повідомлення фрагмента `opt` у сценарії Б; операцію `ЗбереженеСлово.позначитиВідомим()` ЛР5 вивела з переходу 14, а ЛР6 — з повідомлення «`create`» до `ВідомеСлово`. Те, що дві перевірки в різних нотаціях зійшлися на тих самих прогалинах, є непрямим підтвердженням, що прогалини справжні, а не наслідок довільного вибору імен в одній із робіт.

9.2. Що свідомо лишено поза доменною діаграмою класів

Решта позицій переліку — 43 повідомлення груп 1–3 розділу 8.4 ЛР6 і три позиції з групи 4, разом 46 — до діаграми класів **не додано**, і це рішення, а не недоробка. Нижче названо кожну групу й підставу відмови; останнім рядком для повноти наведено й п’ять повідомлень розділу 8.5 ЛР6, які до переліку «потребує додавання» взагалі не входили, бо ЛР6 звіряла їх не з ЛР2, а з діаграмою компонентів ЛР3.

Що ЛР6 просила додати	Де виявлено	Чому свідомо лишено поза доменною діаграмою класів
Межові класи, яким актор надсилає дії: `Оверлей розширення` і `Веб-інтерфейс` — сім повідомлень, а ще два з дев’яти адресовані `КарткаСлова` («натиснути «Зберегти»» і «закрити картку слова»), яку окремо рахує рядок 3 цієї таблиці, уже як отримувача від керівних класів — разом 9 повідомлень	ЛР6, група 1 розділу 8.4	Це елементи подання (boundary). Вони не мають стану, який система пам’ятає між сеансами, тому Фільтр 1 Розділу 1 цієї роботи відсікає їх так само, як відсік `КарткаСлова` в таблиці Розділу 2.3. Їхнє місце — компоненти 1 і 3 діаграми ЛР3, де вони й присутні
Сім керівних класів-делегатів компонентів ЛР3 — `КонтролерОверлея`, `КонтролерВебІнтерфейсу`, `ФасадАрі`, `СервісТарифів`, `СервісНавчання`, `СервісКонтенту`, `ШлюзЗовнішніхСервісів` — 28 повідомлень	ЛР6, група 2 розділу 8.4 і таблиця розділу 8.3	Це процеси, а не сутності. Розділ 2.3 цієї роботи вже відкинув трьох таких самих кандидатів (`МенеджерСловника`, `СервісПерекладу`, `КонтролерЧиталки`), а розділ «Критичний аналіз ШІ-чернетки» назвав їх однією з шести помилок чернетки — функціональною декомпозицією, замаскованою під модель даних. Додати їх зараз означало б повернути в модель саму помилку, яку робота виправила. Делегування «компонент → доменний

		об’єкт», яке описує таблиця 8.3 ЛР6, уже виражене діаграмою компонентів ЛР3
Два тимчасові межові класи `ОверлейРепліки` і `КарткаСлова` — 6 повідомлень, зокрема `«create»`, `показати()` і `«destroy»`	ЛР6, група 3 розділу 8.4	Екранні подання з часом життя однієї репліки або одного кліку. `КарткаСлова` відкинута ще в таблиці Розділу 2.3 цієї роботи з підставою «нічого не зберігає»; ЛР6 цю підставу лише підтвердила, показавши створення й знищення об’єкта в межах однієї ітерації фрагмента `loop`. Клас, який гарантовано не переживає свого фрагмента, не є сутністю предметної області
Обробники асинхронних відповідей шару зовнішніх сервісів: `перекладена частина` і `розбір(JSON)`	ЛР6, група 4 розділу 8.4	Отримувач обох — керівний клас `ШлюзЗовнішніхСервісів` (компонент 8), якого в доменній моделі немає з тієї самої підстави, що й решту контролерів. Самі результати модель зберігає, і саме для цього в розділі 9.1 додано операції `КешПерекладу.зберегти()` і `КешПоясснень.зберегти()`: прогалиною був запис у кеш, а не відсутність класу-шлюза
Розкриття типу `Обробник` і операція `змінившсяЧас(позиціяМс: Long): void`	ЛР6, група 4 розділу 8.4	`Обробник` — контракт зворотного виклику між плеєром і оверлеєм, а отримувач самої операції — `Оверлей розширення`, тобто межовий клас. Тип лишається типом параметра `ДжерелоСубтитрів.спостерігатиНавігацію()` на тій самій підставі, на якій `Реплика` лишається типом значення (Розділ 2.3): він не має ні ідентичності, ні власного життєвого циклу
П’ять повідомлень до зовнішніх систем і до вузла сховища (до переліку з 61 позиції не входять)	ЛР6, розділ 8.5	Сервіс субтитрів YouTube, сервіс перекладу, сервіс ШІ та вузол PostgreSQL 17 + pgvector описані діаграмами компонентів

		і розгортання, а не діаграмою класів предметної області. ЛР6 і перевіряла їх не за ЛР2, а за залежностями ЛР3

Коротко: доменна модель відповідає на питання «що система пам’ятає», а модель проєктування — на питання «хто ці дані рухає». Діаграма послідовності за своєю природою показує саме другий бік, тому її повідомлення масово не мають відповідників у ЛР2 — і 43 позиції з 54 не є дефектом ЛР2, а є доказом, що межа між двома моделями проведена й дотримана.

9.3. Одна позиція, що не потребує ні додавання, ні винесення

Повідомлення `отриматиСтан()`, адресоване об’єктові `розписання : СтанПовторення` у сценарії Б, ЛР6 внесла в групу 4 з приміткою «атрибут `стан: СтанEnum` є, геттера немає». Операцію **не додано**, і підстава тут інша, ніж у розділі 9.2: атрибут `СтанПовторення.стан` оголошений публічним (+), а за правилом видимості з Розділу 3.1 це означає, що значення читається ззовні безпосередньо. Окремий геттер при публічному атрибуті не додає моделі жодного факту — він лише дублює вже оголошений доступ. Тому повідомлення `отриматиСтан()` вважається покритим наявним елементом моделі, а не прогалиною.

9.4. Оновлені підрахунки

Усі числа нижче отримані підрахунком за фактичним вмістом файлів `classes.puml`, `classes_view1.puml`, `classes_view2.puml` і `classes_view3.puml`, а не перенесені з попередньої редакції.

Показник	До узгодження	Після узгодження	Зміна
Елементів діаграми: класи, інтерфейси, фабрика	23	23	нових класів не додано
Атрибутів усього	106	106	без змін
Методів, тобто операцій, усього	63	73	+10
Зв’язків усього	25	26	+1
Композицій / агрегацій / асоціацій / узагальнень / реалізацій / залежностей	9 / 2 / 7 / 2 / 2 / 3	9 / 2 / 7 / 2 / 2 / 4	+1 залежність `ЗбереженеСлово ..> ВідомеСлово : <<create>>`
Перелічуваних типів у таблиці розділу 3.2	12	13	+1, а саме `РезультатВправи`
Значень у `СтанEnum`	4	5	+1, а саме `відкладена`

Розподіл десяти нових операцій по класах:

Клас	Методів до	Методів після	Атрибутів
`ЗбереженеСлово`	3	7	7, без змін
`СтанПовторення`	3	6	6, без змін
`КешПерекладу`	2	3	6, без змін
`КешПоясснень`	2	3	6, без змін
`ДжерелоСубтитрівYouTube`	2	3	4, без змін

Решта 18 елементів діаграми не змінилася зовсім. Вимогу методички «щонайменше 3 атрибути й 2 методи на клас» усі 21 клас, як і раніше, витримують; після доповнення найповнішим за операціями став `ЗбереженеСлово` — 7 методів при 7 атрибутах.

Доповнення зачепили два часткові види з трьох, і їх перерендеровано разом із повною діаграмою.

Файл	Що змінилося	Перерендеровано
`classes.puml` → `classes.svg`, `classes.png`	усі десять операцій	так
`classes_view1.puml` → `.svg`, `.png`	нічого: жоден клас виду 1 не доповнювався	так, для однаковості комплекту
`classes_view2.puml` → `.svg`, `.png`	`ЗбереженеСлово` плюс чотири, `СтанПовторення` плюс три, `КешПоясснень` плюс один	так
`classes_view3.puml` → `.svg`, `.png`	`КешПерекладу` плюс один, `ДжерелоСубтитрівYouTube` плюс один	так

Твердження Розділу 5 про те, що види 1 і 2 є точними підмножинами повної діаграми, лишається чинним: кожна з десяти нових операцій присутня на тому виді, куди входить її клас. Одна відмінність стосується не доповнень, а масштабу й описана в Розділі 5.3: на виді 3 операції інтерфейсу, згорнуті на рисунку 1 розділювачем `.. решта операцій інтерфейсу ..`, виписані повністю, тому в `ДжерелоСубтитрівYouTube` там сім явних рядків проти трьох, а в `ПарсерPDF` — чотири проти двох. Доданих операцій це не змінює. Нове відношення `ЗбереженеСлово ..> ВідомеСлово` на жодному з видів не показане, бо його кінці розведені між видом 2 (`ЗбереженеСлово`) і видом 1 (`ВідомеСлово`); на покриття п’яти типів відношень це не впливає — залежність `<<create>>` вид 3 показує двічі.

Висновок: із 61 позиції, яку ЛР5 і ЛР6 адресували діаграмі класів, доменними прогалинами виявилися чотирнадцять, і закриває їх дванадцять доповнень — десять операцій і дві позиції щодо перелічуваних типів (дванадцять, а не чотирнадцять, бо дві позиції збіглися: ЛР5 і ЛР6 вивели їх незалежно одна від одної). Усі десять операцій додано у вже наявні класи, дві позиції — у таблицю перелічуваних типів Розділу 3.2; жодного нового класу не створено. Нове відношення додано одне й не на вимогу переліку, а для внутрішньої несуперечності: залежність `ЗбереженеСлово ..> ВідомеСлово : <<create>>`, без якої дві з доданих операцій повертали б об’єкт класу, з яким у їхнього класу не було б на діаграмі жодного зв’язку. Сорок шість позицій свідомо лишено поза моделлю, бо вони описують керівні й межові класи рівня проєктування, яким у моделі предметної області не місце; межу «доменна модель проти моделі проєктування» не перенесено, а підтверджено — Розділ 2.3 відкидав цих самих кандидатів ще до побудови ЛР5 і ЛР6. Ще одну позицію, геттер `отриматиСтан()`, визнано непотрібною, бо відповідний атрибут `стан: СтанEnum` оголошений публічним. Загальні підрахунки змінилися у двох рядках: методів стало 73 замість 63, зв’язків — 26 замість 25.

Розділ 10. Відповіді на питання для самоконтролю

10.1. Чим відрізняється агрегація від композиції з погляду життєвого циклу об’єктів

Композиція означає спільний життєвий цикл: частина не може існувати без цілого й видаляється разом із ним, а належати вона може лише одному цілому. Агрегація означає, що ціле лише «має» частину: частина переживає зникнення цілого й може належати кільком цілим або міняти власника. Приклад композиції з цієї роботи — `ЗбереженеСлово ◆ СтанПовторення`: розписання повторень описує конкретну картку слова й разом з нею зникає. Приклад агрегації — `ТемаСловника ◇ ЗбереженеСлово`: видалення теми «Кулінарія» не видаляє слів, вони переходять у службову тему «Різне».

10.2. Що означає множинність `0..1` на асоціативному зв’язку

`0..1` означає «нуль або один», тобто повністю опційний зв’язок: об’єкт або має рівно одного партнера, або не має жодного. У цій роботі така множинність стоїть на **чотирнадцяти** кінцях зв’язків — п’ять разів з боку `Користувач` у композиціях з nullable власником і дев’ять разів у решті відношень. Нижче — три найпоказовіші з чотирнадцяти. `Користувач "1" *-- "0..1" Підписка`: базовий тариф існує без жодного рядка підписки, тому нуль тут — нормальний стан, а не помилка даних.

ЗбереженеСлово "1" *-- "0..1" СтанПовторення: слово, яке ще жодного разу не потрапляло в сесію повторень, розписання не має. ОцінкаРівня "0..1" -- "0..*" ВідомеСлово: слово, позначене відомим вручну в UC07, не походить із жодного тесту обсягу словника, і саме цей нуль відрізняє пряму відмітку від слова, виведеного за частотним рангом.

10.3. У чому різниця між узагальненням і реалізацією на діаграмі класів

Узагальнення (суцільна лінія з порожнім трикутником) — це відношення «є різновидом»: нащадок успадковує і структуру, і поведінку предка, і підмінює його всюди, де той очікується. Реалізація (пунктирна лінія з порожнім трикутником) — це відношення «виконує контракт»: клас зобов'язується мати оголошені операції, але не успадковує жодного поля. На цій діаграмі Відео є різновидом Матеріал — у нього є успадковані назва, зрозумілість, картаЧастот. А ДжерелоСубтитрівYouTube не є різновидом нічого: у трьох реалізацій ДжерелоСубтитрів немає жодного спільного поля, бо одна працює з мережевим запитом до платформи, друга — із завантаженим файлом, третя — із чергою розпізнавання.

10.4. Чому клас позначається курсивом і що це означає для створення екземплярів

Курсив (або стереотип {abstract}) означає, що клас абстрактний: власних екземплярів він не має й створений лише для успадкування. У цій моделі абстрактним є Матеріал: «просто матеріалу» в системі не існує, існують Відео і Документ. Практичний наслідок — у Матеріал є абстрактний метод джерелоТексту(), тіла якого на цьому рівні написати неможливо: у відео це реплики з часовими мітками, у документі — сторінки. Спроба створити «просто Матеріал» дала б об'єкт, у якого цей метод немає реалізації, тобто об'єкт, із яким не працює жоден сценарій ЛР1.

10.5. Яку роль відіграють модифікатори видимості і чому вони важливі вже на етапі моделювання

Видимість — це перший документ про те, що в сутності є контрактом, а що деталлю реалізації. На етапі моделювання вона дає три практичні відповіді. Перша: що взагалі показувати в інтерфейсі — усе публічне в Підписка (статус, кінецьПеріоду) справді бачить користувач, а приватний id — ні. Друга: що має право змінювати лише сама система — Матеріал.картаЧастот приватна, бо перераховується при завантаженні, і дозвіл редагувати її ззовні зробив би оцінку зрозумілості недостовірною. Третя: де пролягає межа спадкування — єдиний захищений атрибут Матеріал.id показує, що саме нащадки мають бачити, а що ні. Без цих рішень діаграма класів перетворюється на малюнок таблиць бази даних.

10.6. Які ризики виникають, якщо покластися на ШІ-згенерований перелік класів без перевірки

Головний ризик — діаграма виглядатиме бездоганно, але описуватиме не цю систему. ШІ не знає рішень, які не записані в тексті: що транскрипт спільний і без власника, що тема словника видаляється через `SET NULL`, що межі сторінок дає лише PDF. Тому його чернетка систематично помиляється у двох напрямках: вона робить усе «частиною» головного класу (бо так симетричніше) і перетворює процеси на класи з назвами на «-менеджер» (бо так виглядає солідніше). Обидві помилки непомітні на око й вилізуть лише на етапі проєктування схеми бази даних. Конкретні шість помилок саме цієї чернетки й виправлення до них наведені в наступному розділі.

Висновок: відповіді спираються на власну модель, а не на приклад методички: кожне теоретичне означення проілюстроване відношенням, яке справді є на побудованій діаграмі.

Критичний аналіз ШІ-чернетки

Чернетку діаграми класів згенеровано ШІ-асистентом за текстовим описом предметної області й переліком варіантів використання ЛР1, після чого проведено ручну звірку з логікою сервісу, зі схемою даних і з означеннями розділу 1.2 методички. Нижче — шість реальних помилок саме цієї чернетки.

Помилка ШІ-чернетки	Чому це помилка	Як виправлено
Спільні довідники прив’язані до `Користувач` композицією. Чернетка послідовно зробила кожен клас «частиною» користувача, бо так симетричніше: і `Транскрипт`, і `СловниковаСтаття` отримали суцільний ромб від акаунта	Доріжка субтитрів, переклад доріжки, пояснення ШІ й стаття довідника спільні: вони зберігаються за хеш-ключем вмісту, а не за власником. Композиція тут знищила б обидва сенси рішення — приватність (із записів неможливо дізнатися, хто що дивився) і економію (другий глядач того самого ролика не платить ні грошима, ні квотою). Крім того, при видаленні акаунта каскад забрав би доріжку, якою користуються інші	Усі такі класи винесено в окремий пакет «Спільні довідники без власника» — `Транскрипт`, `РоботаРозпізнавання`, `КешПерекладу`, `КешПояснень`, `ПереказДоріжки`, `СловниковаСтаття`; зв’язки замінено на асоціації без ромба. Рішення пояснено в Розділі 6.1 і приміткою на самій діаграмі
Обидві агрегації намальовано композиціями. ШІ побачив у описі `ON DELETE CASCADE`	Фізичний каскад у базі даних і композиція в UML — різні твердження. За порівняльною	Обидва зв’язки замінено на агрегації. Для теми це чиста агрегація з кратністю `0..1 —

для `Матеріал → ЗбереженеСлово` і поставив суцільний ромб; для `ТемаСловника → ЗбереженеСлово` він вважав слова структурною частиною теми	таблицею методички частина може належати лише одному цілому в композиції, а `ЗбереженеСлово` вже є частиною `Користувач`: два суцільні ромби на одному класі роблять модель суперечливою. У випадку теми тест методички дає ще й пряму відповідь: видалення теми не видаляє слів, а переводить їх у службову тему «Різне» (`ON DELETE SET NULL`), і слово зазвичай спочатку існує без теми — нерозібрані слова є входом UC15. Композиція означала б, що словник не може містити негрупованих слів	0..*`, а службову тему відображено атрибутом `ТемаСловника.службова: Boolean`. Для матеріалу агрегацію лишено з кратністю `1 — 0..*` і з прямим зазначенням у Розділі 7 (рядок 10), що частина цілого не переживає й що ромб порожній лише через заборону двох композитів
Кратність `1` з боку «цілого» скрізь, де є зовнішній ключ. Чернетка поставила `1` у всіх вісьмох композиціях від `Користувач` і `1` з боку `ОцінкаРівня` в асоціації з `ВідомеСлово`	Обидві групи кратностей суперечать тому, як система справді працює. У схемі власник рядка nullable в налаштуваннях, історії матеріалів, словнику, профілі лексики й історії тестів: рядки, накопичені до появи входу, лишаються без власника. `1` тут означало б, що таких рядків не буває, — а вони є, і механізм їх прив’язки описаний окремо. З `ОцінкаРівня` ще гірше: `1` читалося б як «кожне відоме слово походить із тесту», тоді як `ПоходженняEnum` має значення «пряма відмітка користувача», і це типовий шлях слова в профіль (UC07 — самостійний варіант використання)	Виправлено на `0..1` у п’яти композиціях із nullable власником і на `0..1` з боку `ОцінкаРівня`; там, де власник у схемі обов’язковий (підписка, лічильники, теми словника), `1` збережено. Пояснення — у вступі Розділу 6, у таблиці відношень і примітками на діаграмі та на виді 1
Атрибути, яких у системі немає. Чернетка дописала `ОцінкаРівня.погрішність: Int` і `ЛічильникВикористання.id: UUID`, бо так «акуратніше»: у кожної сутності має бути `id`, а якщо в тексті згадано «± N слів», то це начебто поле	Фільтр 1 цієї ж роботи питає, чи система пам’ятає значення. Похибка оцінки не зберігається: це фіксована константа дизайну («± 400 за базовою вибіркою, ± 200 після уточнення межі»), тобто похідне значення, а не стан. У лічильника витрат	`погрішність` прибрано з атрибутів і замінено методом `+ похибка(): Int` — похідне значення. У `ЛічильникВикористання` знято `id`, а `день` і `вид` позначено стандартним UML-модифікатором `{id}` як

	технічного ідентифікатора немає зовсім: рядок ідентифікується парою «день + вид витрати» в межах власника, що сам Розділ 6 і описує. Зайвий атрибут небезпечніший за відсутній: він обіцяє поле, якого ніхто не заповнює	ідентифікуючу пару; примітка на діаграмі це пояснює
`ДжерелоСубтитрів` — абстрактний клас, а `ДжерелоСубтитрівYouTube` — його нащадок через узагальнення. Чернетка не розрізнила реалізацію й спадкування	Трьох реалізацій не об’єднує жодне спільне поле: одна працює з мережевим запитом до платформи, друга — із завантаженим файлом `.srt`, третя — з чергою розпізнавання. Спільним є лише контракт, а підстановку виконує фабрика за платформою — це Strategy, а не ієрархія типів. Абстрактний клас тут нав'язав би нащадкам стан, якого в них немає, і зламав би можливість додати адаптер нової платформи без зміни предка	Оголошено `interface ДжерелоСубтитрів` з шістьма операціями; зв’язки замінено на реалізацію (`◁┈┈`); додано `ФабрикаДжерелСубтитрів` із залежністю `<<create>>`. Абстрактним класом у моделі лишився `Матеріал`, де спільний стан справді є
Клас-процеси замість сутностей: `МенеджерСловника`, `СервісПерекладу`, `КонтролерЧиталки`, `ОбробникПідписки`. Чернетка видала їх із повними наборами методів	Жоден із них не має стану, який система пам’ятає між сеансами, — отже, на діаграмі класів предметної області їм не місце. Це функціональна декомпозиція, замаскована під модель даних: вона робить діаграму більшою, але не додає жодного факту про предметну область. Їхнє правильне місце — діаграма компонентів ЛР3	Усі чотири прибрано; їхню поведінку роздано сутностям як методи (`ЗбереженеСлово.схожіЗаЗмістом()`, `Транскрипт.перекласти()`, `Документ.сторінка()`, `Підписка.стеля()`), а самі назви перенесено в перелік майбутніх компонентів ЛР3

Висновок: чернетка ШІ дала корисний чорновий перелік сутностей за кілька хвилин, але жодне з шести виправлень не вимагало знання UML — усі шість вимагали знання саме цього сервісу та його схеми даних. Три помилки з шести — про той самий нахил: усе зробити частиною користувача (зайві композиції, два суцільні ромби на одному класі, кратність `1` з боку власника). Найнебезпечнішою виявилася не плутанина агрегації з композицією, яку видно за тестом методички, а саме кратність: вона виглядає бездоганно й суперечить системі лише тоді, коли звірити її з конкретною колонкою схеми.

Висновки

У роботі побудовано діаграму класів предметної області «Півот» — вебсервісу й браузерного розширення для вивчення іноземної мови на основі власного контенту користувача. Діаграма містить 23 елементи: 17 основних класів предметної області (серед них обов’язковий абстрактний клас `Матеріал`), один резервний клас `ПереказДоріжки`, два інтерфейси `ДжерелоСубтитрів` і `ПарсерДокумента`, два класи-реалізації та одну фабрику. Кожен із 21 класу має щонайменше три атрибути з явною видимістю й типом і щонайменше два методи; усього на діаграмі 106 атрибутів і 73 операції, що втричі перевищує мінімум методички в 6–8 класів. Оскільки повна діаграма на аркуші читається погано, додатково побудовано три часткові види: види 1 і 2 — точні підмножини повної діаграми, вид 3 — підмножина плюс чотири сірі класи реалізацій, які прямо названо такими, що в модель не входять.

Представлено всі п’ять типів відношень із теоретичної частини: дев’ять композицій, дві агрегації, сім асоціацій з явною множинністю обох кінців, два узагальнення й дві реалізації інтерфейсу, а також чотири залежності, три з них зі стереотипом `<<create>>` — разом 26 зв’язків. Кожне з одинадцяти відношень «ціле-частина» обґрунтовано письмово за тестом розділу 1.2.5 методички, і одне з одинадцяти свідомо лишено як видиме розходження: `Матеріал` ◇ `ЗбереженеСлово` за першим критерієм було б композицією, бо видалення матеріалу справді забирає і слова, і їхні розписання, але композит у частини може бути лише один, і моделлю ним обрано `Користувач`. Це розходження не приховано, а описано в Розділі 7 і приміткою просто на зв’язку.

Окремо перевірено кратності, бо саме в них найлегше збрехати непомітно. П’ять композицій від `Користувач` мають з боку власника `0..1`, а не `1`: у схемі рядок може бути ще не прив’язаним до акаунта, і нуль позначає цей перехідний стан. Там, де власник обов’язковий, збережено `1`. Агрегація від `Матеріал` має кратність `1`, бо картка слова без матеріалу не створюється взагалі. Асоціація `ОцінкаРівня` – `ВідомеСлово` має `0..1` з боку оцінки, бо слово, позначене відомим вручну, з жодного тесту не походить. Кожне з цих рішень читається з моделі й перевіряється одним питанням до схеми даних.

Головним змістовним результатом роботи є не сама діаграма, а три рішення, які вона зробила видимими. Перше — абстрактний клас `Матеріал` з абстрактним методом `джерелоТексту()`: він дозволяє оцінці зрозумілості, збереженню слова й розбору речення працювати однаково з відео й з документом, тому ці сценарії не

довелося описувати двічі; нащадків рівно два, бо нащадок з’являється там, де відрізняється реалізація цього методу, а стаття за посиланням дає той самий текст, що й файл. Друге — інтерфейс `ДжерелоСубтитрів` із фабрикою: він перетворює обмеження «лише YouTube» з вимушеного спрощення на архітектурне рішення про межі роботи, бо нова платформа підключається реалізацією контракту, а не зміною оверлея. Третє — шість класів-довідників без власника: відсутність у них будь-якого зв’язку з `Користувач` одночасно забезпечує приватність (з даних неможливо відновити, хто що дивився) і економію на двох найдорожчих операціях — розпізнаванні мовлення й перекладі доріжки.

Узгодженість із Лабораторною роботою №1 перевірено в трьох зрізах. Усі 18 варіантів використання дістали класи, причому жоден не трасується на клас, відсутній на оцінюваній діаграмі, і всі чотири платні варіанти використання однаково трасуються на гейт `Підписка`. Усі ключові іменники трьох повних текстових специфікацій (UC04, UC05, UC12) знайшли місце серед класів, атрибутів або методів. Відображення акторів на класи свідомо не зроблено взаємно однозначним: актор — це роль, тому «Користувач з підпискою» став не класом-нащадком, а станом `Користувач.тариф()`. Використання ШІ дало корисний чорновий перелік сутностей за кілька хвилин, але всі шість виправлень вимагали знання саме цього сервісу та його схеми даних, а не знання UML: ШІ систематично робить усе частиною головного класу, ставить `1` там, де колонка nullable, і дописує атрибути, яких ніхто не зберігає. Побудована модель прямо готує наступні роботи: перелічення `СтанEnum` і `СтанРоботиEnum` уже містять готові набори станів для діаграм станів картки слова й роботи з розпізнавання мовлення, а фактичну, неповну відповідність пакетів класів до восьми компонентів ЛР3 наведено в Розділі 8.4.

Модель довелося уточнити вже за результатами двох наступних робіт, і Розділ 9 описує це уточнення повністю. Діаграма станів ЛР5 і комплект діаграм послідовності ЛР6 разом назвали 61 позицію, якої в ЛР2 не було; після розбору чотирнадцять із них виявилися справжніми прогалинами доменної моделі й закриті дванадцятьма доповненнями до вже наявних класів (дві позиції збіглися, бо виведені двома роботами незалежно) — це десять операцій (`ЗбереженеСлово.створити()`, `зарахуватиДоЗнайомих()`, `позначитиВідомим()`, `видалити()`; `СтанПовторення.короткийКрок()`, `відкласти()`, `продовжити()`; `КешПерекладу.зберегти()`, `КешПояснень.зберегти()`, `ДжерелоСубтитрівYouTube.прийнятиДоріжку()`) і дві позиції щодо перелічуваних типів (`СтанEnum.відкладена` та склад типу `РезультатВправи`). Через це кількість

операцій на діаграмі зросла з 63 до 73, а кількість зв’язків — з 25 до 26: дві з доданих операцій повертають об’єкт `ВідомеСлово`, тому за конвенцією моделі до діаграми додано залежність `ЗбереженеСлово ..> ВідомеСлово : <<create>>`. Кількість елементів (23) і атрибутів (106) не змінилася зовсім: жодного нового класу створювати не довелося. Решту 46 позицій свідомо лишено поза діаграмою, бо вони описують керівні й межові класи рівня проєктування, яким у моделі предметної області не місце, а ще одну — геттер `отриматиСтан()` — визнано непотрібною, бо відповідний атрибут оголошений публічним. Це і є головний підсумок перевірки: прогалин у ЛР2 було мало й усі вони дрібні, а масова відсутність відповідників у ЛР6 виявилася не дефектом, а підтвердженням, що межу «доменна модель проти моделі проєктування» ця робота провела правильно ще в Розділі 2.3.